

Detección de anomalías y técnicas de agrupamiento

Aplicación de aprendizaje no supervisado al conjunto de datos de cardiotocografía (CTG)

Asignatura: Aprendizaje Automático

Programa: Maestría en Inteligencia Artificial

Autor: Abel Meneses Yupanqui

Conjunto de datos: CTG.csv, con 2 126 registros de monitoreo fetal

Entorno: Python 3.14 con pandas, scikit-learn, SciPy, matplotlib y seaborn

Código fuente:

<https://github.com/amenesesy/actividad-ml-ctg>

Todo el código, las figuras y los resultados numéricos que aparecen en este informe proceden de una única ejecución del cuaderno ML_Actividad_CTG.ipynb, con la semilla aleatoria fijada en 42 para que los resultados sean reproducibles. El repositorio enlazado contiene el cuaderno ejecutado, los scripts que lo generan y las quince figuras en su resolución original.

Contenido

0. Configuración del entorno
1. El conjunto de datos
2. Análisis descriptivo de los datos
3. Tratamiento de los valores faltantes
4. Preparación de la matriz de características
5. Detección de anomalías
6. Técnicas de agrupamiento
7. Ventajas y desventajas de cada modelo
8. Conclusiones

Referencias

El informe reproduce íntegramente el desarrollo del cuaderno. Para cada etapa se presenta primero la explicación metodológica, después el código Python comentado, a continuación la salida que produce y, por último, la interpretación de los resultados obtenidos.

Las salidas de consola más extensas aparecen recortadas para respetar la extensión máxima que fija el enunciado. La versión completa, con todas las salidas y las figuras a resolución original, está disponible en el repositorio público <https://github.com/ameneses/actividad-ml-ctg>.

0. Configuración del entorno

El cuaderno está preparado para ejecutarse en dos entornos sin ningún cambio manual. En una instalación local basta con tener el archivo **CTG.csv** junto al cuaderno. En Google Colab, al que se accede mediante el distintivo que encabeza el documento, la primera celda detecta el entorno, comprueba que estén las librerías necesarias, instala las que falten y descarga el conjunto de datos desde el repositorio público del trabajo. La ejecución completa tarda alrededor de dos minutos.

La segunda celda reúne en un solo lugar todas las librerías del ecosistema científico de Python que se van a utilizar y fija la semilla aleatoria. Esto último no es un detalle menor: Isolation Forest, K-Means y el experimento de imputación son algoritmos estocásticos, de manera que sin una semilla fija producirían cifras distintas en cada ejecución y ninguna de las conclusiones sería verificable.

Listado 1. Celda 0.1. Preparacion del entorno y compatibilidad con Google Colab

```
# Celda 0.1. Preparacion del entorno y compatibilidad con Google Colab.
# Objetivo: que el cuaderno se ejecute igual en local que en Colab. Se detecta
# el entorno, se instalan las librerías que falten y se descarga el conjunto
# de datos del repositorio publico si no esta presente en el directorio.
# Salidas: la constante EN_COLAB y el archivo CTG.csv disponible en el disco.
import importlib.util
import pathlib
import subprocess
import sys
import urllib.request
# Deteccion del entorno. En Colab existe el paquete google.colab; fuera de
# Colab ni siquiera existe el paquete contenedor google, así que la deteccion
# se hace con un import protegido y no con find_spec, que ahí fallaría.
try:
```

```

import google.colab # noqa: F401
EN_COLAB = True
except ImportError:
    EN_COLAB = False
# Direccion base del repositorio publico del trabajo, desde donde se descargan
# los datos cuando el cuaderno se abre en un entorno recién creado.
REPO = "https://raw.githubusercontent.com/ameneses/actividad-ml-ctg/main"
# Dependencias del cuaderno, como pares (modulo que se importa, paquete de pip).
DEPENDENCIAS = [
    ("numpy", "numpy"),
    ("pandas", "pandas"),
    ("matplotlib", "matplotlib"),
    ("seaborn", "seaborn"),
    ("sklearn", "scikit-learn"),
    ("scipy", "scipy"),
]
faltantes = [pip for modulo, pip in DEPENDENCIAS
              if importlib.util.find_spec(modulo) is None]
if faltantes:
    print("Instalando dependencias que faltan:", ", ".join(faltantes))
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", *faltantes],
                   check=True)
else:
    print("Todas las dependencias estan disponibles.")
# El conjunto de datos se descarga solo si no esta ya en el directorio actual.
if not pathlib.Path("CTG.csv").exists():
    print("Descargando CTG.csv desde el repositorio del trabajo...")
    urllib.request.urlretrieve(REPO + "/CTG.csv", "CTG.csv")
print("Entorno de ejecucion:", "Google Colab" if EN_COLAB else "instalacion local")
print("Version de Python   :", sys.version.split()[0])
print("Conjunto de datos   :", "CTG.csv disponible"
      if pathlib.Path("CTG.csv").exists() else "NO disponible")

```

```

Todas las dependencias estan disponibles.
Entorno de ejecucion: instalacion local
Version de Python   : 3.14.0
Conjunto de datos   : CTG.csv disponible

```

Listado 2. Celda 0.2. Importacion de librerias y configuracion global

```

# Celda 0.2. Importacion de librerias y configuracion global.
# Objetivo: cargar todas las dependencias en un unico lugar, fijar la semilla
# aleatoria y homogeneizar el estilo de los graficos.
# Salidas: las constantes globales SEMILLA y DIR_FIGURAS, y el diccionario RES
# en el que cada seccion va depositando sus resultados numericos.
# Utilidades del sistema
import json                # serializacion de los resultados a disco
import pathlib             # manejo de rutas independiente del sistema
import warnings            # control de avisos no criticos
# Manipulacion de datos
import numpy as np         # algebra vectorial y generacion aleatoria
import pandas as pd        # estructuras tabulares (DataFrame)
# Visualizacion
import matplotlib.pyplot as plt # motor de graficos de bajo nivel
import seaborn as sns       # graficos estadisticos de alto nivel
# Preprocesamiento
from sklearn.preprocessing import StandardScaler # z = (x - mu) / sigma
from sklearn.impute import SimpleImputer, KNNImputer # estrategias de imputacion
# Deteccion de anomalias
from sklearn.ensemble import IsolationForest # aislamiento por particiones
from sklearn.neighbors import LocalOutlierFactor # densidad local relativa
from sklearn.covariance import MinCovDet # covarianza robusta (MCD)
# Agrupamiento
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
# Reduccion de dimensionalidad, vecindades y metricas
from sklearn.decomposition import PCA
from sklearn.neighbors import NearestNeighbors
from sklearn.metrics import (
    silhouette_score, # cohesion frente a separacion (validacion interna)
    silhouette_samples, # silueta calculada por observacion
    davies_bouldin_score, # razon dispersion/separacion (menor es mejor)
    calinski_harabasz_score, # razon de varianzas entre/intra (mayor es mejor)
    adjusted_rand_score, # concordancia con etiquetas (validacion externa)
    normalized_mutual_info_score, # informacion mutua normalizada
)
from scipy import stats # pruebas estadisticas (Kolmogorov-Smirnov, chi2)
# Configuracion global del cuaderno
SEMILLA = 42 # unica fuente de aleatoriedad
np.random.seed(SEMILLA)
warnings.filterwarnings("ignore") # silencia avisos de version
pd.set_option("display.width", 140) # ancho de impresion de los DataFrame
pd.set_option("display.max_columns", 50)
sns.set_theme(style="whitegrid", palette="deep") # estilo homogeneo de figuras
plt.rcParams["figure.dpi"] = 110
plt.rcParams["savefig.dpi"] = 200 # resolucion suficiente para imprimir
plt.rcParams["savefig.bbox"] = "tight"
# Tipografia holgada dentro de las figuras. Las figuras se disenar compactas y
# con letra grande para que sigan siendo legibles cuando el informe en PDF las
# reduce al ancho de la caja de texto.
plt.rcParams["font.size"] = 11
plt.rcParams["axes.titlesize"] = 12
plt.rcParams["axes.labelsize"] = 11
plt.rcParams["xtick.labelsize"] = 10
plt.rcParams["ytick.labelsize"] = 10
plt.rcParams["legend.fontsize"] = 10
DIR_FIGURAS = pathlib.Path("figuras") # carpeta destino de las figuras
DIR_FIGURAS.mkdir(exist_ok=True)
# Diccionario acumulador. Cada seccion deposita aqui sus resultados numericos
# para que el informe en PDF se construya despues a partir de datos reales y no
# de cifras transcritas a mano.
RES = {}
def guardar(nombre):
    """Guarda la figura activa de matplotlib en figuras/<nombre>.png y la muestra.
    Parametros
    -----
    """

```

```

nombre : str
    Nombre del archivo destino, sin extension ni ruta.
"""
plt.savefig(DIR_FIGURAS / (nombre + ".png"))
plt.show()
print("Librerías cargadas correctamente. Semilla aleatoria fijada en", SEMILLA)

```

Librerías cargadas correctamente. Semilla aleatoria fijada en 42

1. El conjunto de datos

1.1 Contexto del problema

La cardiotocografía es la prueba de vigilancia fetal más extendida del mundo. Registra de manera simultánea la frecuencia cardíaca del feto y la actividad contráctil del útero durante el embarazo tardío y el trabajo de parto. Su interpretación visual tiene, sin embargo, una concordancia entre observadores notoriamente baja, y por ese motivo desde los años noventa se desarrollaron sistemas de análisis automático capaces de resumir cada trazado en un vector de descriptores numéricos.

El archivo **CTG.csv** es precisamente el resultado de aplicar el sistema SisPorto 2.0 (Ayres-de-Campos et al., 2000) a 2 126 trazados. Cada fila corresponde a un segmento de registro y cada columna a un descriptor calculado de forma automática. Además, tres obstetras etiquetaron cada segmento con dos variables de diagnóstico: **CLASS**, que recoge el patrón morfológico de la frecuencia cardíaca fetal en diez categorías, y **NSP**, que resume el estado del feto en tres niveles, normal, sospechoso y patológico.

1.2 Por qué este conjunto se ajusta a la actividad

El problema encaja de forma natural con las dos técnicas que pide el enunciado. En cuanto a la detección de anomalías, los casos patológicos son por definición minoritarios, apenas el 8,3 % del total, de modo que un buen detector de valores atípicos debería enriquecerse en ellos sin haberlos visto nunca. Eso permite medir de forma objetiva si el detector resulta útil y no solo si es matemáticamente correcto. En cuanto al agrupamiento, si la frecuencia cardíaca fetal presenta realmente patrones morfológicos diferenciados, un algoritmo de clustering debería recuperarlos al menos en parte. El contraste posterior con **NSP** cuantifica cuánta de esa estructura es real y cuánta es un artefacto del algoritmo.

Listado 3. Celda 1.1. Carga del conjunto de datos

```

# Celda 1.1. Carga del conjunto de datos.
# Se usa la copia local que dejo preparada la celda 0.1 y, si por lo que fuera
# no estuviera, se recurre al archivo original publicado por el curso.
# Salidas: df_bruto, con los datos tal y como vienen del archivo.
RUTA_LOCAL = pathlib.Path("CTG.csv")
URL_CURSO = ("https://raw.githubusercontent.com/OscarJimenezFlores/ML/"
            "refs/heads/main/Data/CTG.csv")
origen = RUTA_LOCAL if RUTA_LOCAL.exists() else URL_CURSO
df_bruto = pd.read_csv(origen)
print("Origen de los datos      :", origen)
print("Dimensiones (filas, columnas):", df_bruto.shape)
print("\nPrimeras 3 filas:")
df_bruto.head(3)

```

```

Origen de los datos      : CTG.csv
Dimensiones (filas, columnas): (2129, 40)
Primeras 3 filas:
  FileName  Date      SegFile  b      e  LBE  LB  AC  FM  UC  ASTV  MSTV  ALTV  MLTV  DL  DS  DP  DR  Width  \
0 Variab10.txt  12/1/1996  CTG0001.txt  240.0  357.0  120.0  120.0  0.0  0.0  0.0  73.0  0.5  43.0  2.4  0.0  0.0  0.0  0.0  64.0
1 Fmcs_1.txt    5/3/1996  CTG0002.txt   5.0  632.0  132.0  132.0  4.0  0.0  4.0  17.0  2.1  0.0  10.4  2.0  0.0  0.0  0.0  130.0
2 Fmcs_1.txt    5/3/1996  CTG0003.txt  177.0  779.0  133.0  133.0  2.0  0.0  5.0  16.0  2.1  0.0  13.4  2.0  0.0  0.0  0.0  130.0
  Min  Max  Nmax  Nzeros  Mode  Mean  Median  Variance  Tendency  A  B  C  D  E  AD  DE  LD  FS  SUSP  CLASS  NSP
0  62.0  126.0  2.0  0.0  120.0  137.0  121.0  73.0  1.0  0.0  0.0  0.0  0.0  0.0  0.0  0.0  1.0  0.0  9.0  2.0
[Salida recortada: 2 líneas mas. La salida completa esta en el cuaderno del repositorio.]

```

Antes de calcular ningún estadístico conviene documentar qué significa cada columna y, sobre todo, qué papel juega. Sin ese paso el análisis exploratorio se convierte en estadística ciega, porque no hay forma de decidir qué variables son informativas, cuáles son simples identificadores del registro, cuáles duplican información que ya está en otra parte y cuáles son etiquetas que deben quedar apartadas del modelado.

Listado 4. Celda 1.2. Diccionario de variables

```
# Celda 1.2. Diccionario de variables.
# Documenta el significado y el rol de cada una de las 40 columnas del archivo.
# Salidas: el diccionario DICCIONARIO y la tabla legible tabla_dic.
DICCIONARIO = {
    # Identificadores del registro, sin valor descriptivo
    "FileName": ("Identificador", "Nombre del archivo del trazado original"),
    "Date": ("Identificador", "Fecha del registro"),
    "SegFile": ("Identificador", "Nombre del segmento analizado"),
    "b": ("Identificador", "Indice de inicio del segmento en el trazado"),
    "e": ("Identificador", "Indice de fin del segmento en el trazado"),
    # Descriptores de la frecuencia cardiaca fetal
    "LBE": ("Redundante", "Linea de base de la FCF segun el experto medico"),
    "LB": ("Numerica", "Linea de base de la FCF calculada por SisPorto (lpm)"),
    "AC": ("Numerica", "Numero de aceleraciones"),
    "FM": ("Numerica", "Numero de movimientos fetales"),
    "UC": ("Numerica", "Numero de contracciones uterinas"),
    "ASTV": ("Numerica", "% de tiempo con variabilidad anormal a corto plazo"),
    "MSTV": ("Numerica", "Valor medio de la variabilidad a corto plazo"),
    "ALTV": ("Numerica", "% de tiempo con variabilidad anormal a largo plazo"),
    "MLTV": ("Numerica", "Valor medio de la variabilidad a largo plazo"),
    "DL": ("Numerica", "Numero de deceleraciones ligeras"),
    "DS": ("Numerica", "Numero de deceleraciones severas"),
    "DP": ("Numerica", "Numero de deceleraciones prolongadas"),
    "DR": ("Constante", "Numero de deceleraciones repetitivas (siempre 0)"),
    # Descriptores del histograma de la frecuencia cardiaca fetal
    "Width": ("Numerica", "Amplitud del histograma de la FCF"),
    "Min": ("Numerica", "Valor minimo del histograma"),
    "Max": ("Numerica", "Valor maximo del histograma"),
    "Nmax": ("Numerica", "Numero de picos del histograma"),
    "Nzeros": ("Numerica", "Numero de ceros del histograma"),
    "Mode": ("Numerica", "Moda del histograma"),
    "Mean": ("Numerica", "Media del histograma"),
    "Median": ("Numerica", "Mediana del histograma"),
    "Variance": ("Numerica", "Varianza del histograma"),
    "Tendency": ("Ordinal", "Tendencia del histograma (-1 izq., 0 simetrica, 1 der.)"),
    # Codificacion disyuntiva (one-hot) de la etiqueta CLASS
    **{c: ("One-hot de CLASS", "Indicador binario del patron morfologico " + c)
       for c in ["A", "B", "C", "D", "E", "AD", "DE", "LD", "FS", "SUSP"]},
    # Etiquetas de diagnostico, reservadas como verdad de referencia
    "CLASS": ("Etiqueta", "Patron morfologico de la FCF (10 categorias)"),
    "NSP": ("Etiqueta", "Estado fetal: 1=Normal, 2=Sospechoso, 3=Patologico"),
}
tabla_dic = pd.DataFrame(
    [(v, r, d) for v, (r, d) in DICCIONARIO.items()],
    columns=["Variable", "Rol", "Descripcion"],
)
print("Total de columnas documentadas:", len(tabla_dic), "\n")
print("Reparto por rol:")
print(tabla_dic["Rol"].value_counts().to_string())
RES["n_columnas"] = int(df_bruto.shape[1])
tabla_dic
```

```
Total de columnas documentadas: 40
Reparto por rol:
Rol
Numerica      20
One-hot de CLASS  10
Identificador   5
Etiqueta       2
Redundante     1
Constante      1
Ordinal        1
[Salida recortada: 41 lineas mas. La salida completa esta en el cuaderno del repositorio.]
```

El reparto por rol deja ver que el archivo contiene bastante menos información de la que sugieren sus 40 columnas. Cinco de ellas identifican el registro, una duplica a otra, otra es constante, diez son una recodificación de la etiqueta **CLASS** y dos son las propias etiquetas. Solo veintiuna describen realmente el trazado, y en la sección 4 se comprobará que incluso entre esas hay una redundancia exacta.

2. Análisis descriptivo de los datos

El análisis exploratorio persigue tres objetivos concretos antes de aplicar ningún algoritmo. El primero es conocer la escala y la forma de cada variable, porque tanto la detección de anomalías

como el agrupamiento se basan en distancias y son, por tanto, muy sensibles a las unidades de medida y a la asimetría. El segundo es detectar los problemas de calidad, es decir, filas espurias, columnas constantes, duplicados y redundancias que contaminarían los modelos si pasaran inadvertidos. El tercero es cuantificar la redundancia entre variables mediante la matriz de correlaciones, dato que resultará decisivo para saber si conviene reducir la dimensionalidad.

2.1 Estructura del conjunto

Listado 5. Celda 2.1. Estructura general del conjunto de datos

```
# Celda 2.1. Estructura general del conjunto de datos.
# Objetivo: inspeccionar dimensiones, tipos de dato, memoria y duplicados. Es el
# primer control de calidad, y cualquier anomalia estructural debe detectarse
# aqui y no cuando ya estan fallando los modelos.
print("=" * 78)
print("ESTRUCTURA DEL CONJUNTO DE DATOS")
print("=" * 78)
print("Numero de filas      :", df_bruto.shape[0])
print("Numero de columnas   :", df_bruto.shape[1])
print("Memoria ocupada      : %.1f KB" % (df_bruto.memory_usage(deep=True).sum() / 1024))
# Reparto de tipos de dato: distingue las columnas de texto de las numericas.
print("\nTipos de dato presentes:")
print(df_bruto.dtypes.value_counts().to_string())
# Filas completamente duplicadas: en un conjunto de senales clinicas serian
# sospechosas de un error de exportacion.
print("\nFilas exactamente duplicadas:", int(df_bruto.duplicated().sum()))
# Ultimas filas del archivo: es donde suelen aparecer los artefactos de las
# hojas de calculo originales, como filas de totales o separadores en blanco.
print("\nUltimas 4 filas del archivo (primeras 12 columnas):")
print(df_bruto.iloc[-4:, :12].to_string())
```

```
=====
ESTRUCTURA DEL CONJUNTO DE DATOS
=====
Numero de filas      : 2129
Numero de columnas   : 40
Memoria ocupada      : 985.9 KB
Tipos de dato presentes:
float64      37
str           3
[Salida recortada: 8 lineas mas. La salida completa esta en el cuaderno del repositorio.]
```

La inspección de las últimas filas revela el primer hallazgo importante del trabajo, y es que el archivo no termina en un registro clínico. Las tres últimas líneas son residuos de la hoja de cálculo original **CTG.xls**: una fila completamente vacía, una fila con ceros de relleno y una fila de totales que agrega columnas como **FM**, **UC** o **ASTV**. No son pacientes, sino artefactos de exportación. El detalle condiciona por completo la sección 3, porque son exactamente esas tres filas las que generan todos los valores faltantes del conjunto.

2.2 Estadísticos descriptivos de las variables numéricas

Listado 6. Celda 2.2. Estadísticos descriptivos de las variables numericas

```
# Celda 2.2. Estadísticos descriptivos de las variables numericas.
# Objetivo: obtener tendencia central, dispersion y forma de cada variable. A
# los descriptivos clasicos se anaden la asimetria y la curtosis, que son las
# medidas que anticipan como se comportaran los detectores de anomalias
# basados en la hipotesis de normalidad.
# Salidas: el DataFrame desc, con una fila por variable.
# Se excluyen las columnas de texto; el resto son numericas.
num_bruto = df_bruto.select_dtypes(include=[np.number])
# describe() entrega conteo, media, desviacion, minimo, cuartiles y maximo.
desc = num_bruto.describe().T
# Asimetria: 0 indica simetria; un valor absoluto mayor que 1 indica cola marcada.
desc["asimetria"] = num_bruto.skew()
# Curtosis de Fisher: 0 es la normal; por encima de 3 hay colas muy pesadas.
desc["curtosis"] = num_bruto.kurtosis()
# Coeficiente de variacion: dispersion relativa, comparable entre variables.
desc["cv"] = desc["std"] / desc["mean"].replace(0, np.nan)
desc = desc.round(2)
print("Estadísticos descriptivos de las", desc.shape[0], "variables numericas:\n")
print(desc[["count", "mean", "std", "min", "50%", "max", "asimetria", "curtosis"]].to_string())
# Se guardan las variables mas asimetricas, que son las que mas condicionan la
# eleccion del metodo de deteccion de anomalias.
RES["top_asimetria"] = desc["asimetria"].abs().sort_values(ascending=False).head(6).round(2).to_dict()
desc
```

Estadísticos descriptivos de las 37 variables numericas:								
	count	mean	std	min	50%	max	asimetria	curtosis
b	2126.0	878.44	894.08	0.0	538.0	3296.0	0.83	-0.54
e	2126.0	1702.88	930.92	287.0	1241.0	3599.0	0.66	-0.82
LBE	2126.0	133.30	9.84	106.0	133.0	160.0	0.02	-0.29
LB	2126.0	133.30	9.84	106.0	133.0	160.0	0.02	-0.29
AC	2126.0	2.72	3.56	0.0	1.0	26.0	1.66	3.12

FM	2127.0	7.50	39.03	0.0	0.0	564.0	9.46	104.34
UC	2127.0	3.67	2.88	0.0	3.0	23.0	0.94	2.06
ASTV	2127.0	47.01	17.21	12.0	49.0	87.0	-0.01	-1.05

[Salida recortada: 67 líneas mas. La salida completa esta en el cuaderno del repositorio.]

La tabla admite tres lecturas relevantes. La primera es que la columna **count** no es constante: la mayoría de variables tiene 2 126 observaciones válidas, pero unas pocas tienen 2 127 o 2 128, y esa irregularidad es la huella de las filas de artefacto detectadas en el apartado anterior.

La segunda es que las escalas son radicalmente distintas. La línea de base **LB** se mueve en torno a 133 latidos por minuto, **MSTV** en torno a 1,3 y **Variance** alcanza valores de 269. Cualquier algoritmo basado en la distancia euclídea quedaría dominado por las variables de rango grande, de modo que la estandarización no es opcional sino imprescindible, cuestión que se retoma en la sección 4.

La tercera lectura tiene que ver con la forma de las distribuciones. Variables como **DS**, **DP**, **Variance** o **Nzeros** presentan asimetrías muy superiores a 1 y curtosis elevadas, porque son conteos de eventos raros con una enorme acumulación de ceros. Esto anticipa que los métodos de detección de anomalías que asumen normalidad, como el Z-score o la distancia de Mahalanobis, marcarán muchos falsos positivos, y favorece de entrada a los métodos no paramétricos.

2.3 Variables categóricas y frecuencias

Listado 7. Celda 2.3. Variables categoricas: categorias y frecuencias

```
# Celda 2.3. Variables categoricas: categorias y frecuencias.
# Objetivo: listar las categorias de cada variable cualitativa y su frecuencia.
# Se distinguen dos grupos, porque pandas solo reconoce el primero: las
# categoricas almacenadas como texto (FileName, Date, SegFile) y las
# codificadas como numeros (CLASS, NSP, Tendency y las diez indicadoras).
# Tratar estas ultimas como continuas seria un error conceptual, porque la
# media de NSP no significa nada.
# Categoricas almacenadas como texto
cat_texto = df_bruto.select_dtypes(include=["object"]).columns.tolist()
print("Variables categoricas de tipo texto:", cat_texto, "\n")
for col in cat_texto:
    n_cat = df_bruto[col].nunique()
    print(" " + col + ": " + str(n_cat) + " categorias distintas "
          + "(cardinalidad " + str(round(100 * n_cat / len(df_bruto), 1)) + "% de las filas)")
    print(" 5 mas frecuentes: " + str(df_bruto[col].value_counts().head(5).to_dict()))
print("\n" + "-" * 78)
print("La cardinalidad casi maxima confirma que las tres son IDENTIFICADORES")
print("del registro y no variables analizables.")
print("-" * 78 + "\n")
# Categoricas codificadas como numeros
cat_numericas = ["Tendency", "CLASS", "NSP"]
ETIQUETAS_NSP = {1: "Normal", 2: "Sospechoso", 3: "Patologico"}
ETIQUETAS_TEND = {-1: "Desplazado a la izquierda", 0: "Simetrico", 1: "Desplazado a la derecha"}
for col in cat_numericas:
    frec = df_bruto[col].value_counts(dropna=False).sort_index()
    prop = (100 * frec / frec.sum()).round(2)
    tabla = pd.DataFrame({"frecuencia": frec, "porcentaje": prop})
    if col == "NSP":
        tabla.insert(0, "significado", [ETIQUETAS_NSP.get(i, "artefacto") for i in tabla.index])
    if col == "Tendency":
        tabla.insert(0, "significado", [ETIQUETAS_TEND.get(i, "artefacto") for i in tabla.index])
    print("Frecuencias de " + col + " :")
    print(tabla.to_string(), "\n")
# Coherencia de la codificacion one-hot de CLASS
COLS_ONEHOT = ["A", "B", "C", "D", "E", "AD", "DE", "LD", "FS", "SUSP"]
suma_onehot = df_bruto[COLS_ONEHOT].sum(axis=1)
print("Suma por fila de las 10 indicadoras one-hot:",
      suma_onehot.value_counts(dropna=False).to_dict())
print("Al ser constante e igual a 1, se trata de una recodificacion EXACTA de")
print("CLASS: no aportan informacion nueva e introducen una dependencia lineal")
print("perfecta que rompe los metodos basados en la matriz de covarianzas.")
RES["frec nsp"] = df_bruto["NSP"].value_counts(dropna=False).sort_index().to_dict()
```

```
Variables categoricas de tipo texto: ['FileName', 'Date', 'SegFile']
FileName: 352 categorias distintas (cardinalidad 16.5% de las filas)
 5 mas frecuentes: {'S8001034.dsp': 34, 'S7001029.dsp': 33, 'S8001037.dsp': 30, 'S8001038.dsp': 26, 'S7001027.dsp': 24}
Date: 48 categorias distintas (cardinalidad 2.3% de las filas)
 5 mas frecuentes: {'2/22/1995': 240, '5/2/1996': 160, '7/18/1996': 101, '10/3/1996': 92, '5/3/1996': 88}
SegFile: 2126 categorias distintas (cardinalidad 99.9% de las filas)
 5 mas frecuentes: {'CTG0001.txt': 1, 'CTG0002.txt': 1, 'CTG0003.txt': 1, 'CTG0004.txt': 1, 'CTG0005.txt': 1}
-----
La cardinalidad casi maxima confirma que las tres son IDENTIFICADORES
[Salida recortada: 38 líneas mas. La salida completa esta en el cuaderno del repositorio.]
```

El análisis de las categóricas produce cuatro decisiones de modelado. Las variables **FileName**, **Date** y **SegFile** tienen una cardinalidad casi igual al número de filas, lo que las identifica como identificadores del registro; no describen al feto y por tanto se descartan. La variable **NSP** está muy desbalanceada, con un 77,8 % de casos normales, un 13,9 % de sospechosos y un 8,3 % de patológicos, y ese desbalance es justamente lo que convierte al conjunto en un buen banco de pruebas para la detección de anomalías. La variable **CLASS** reparte los datos en diez categorías con frecuencias muy dispares, entre 53 y 579 casos. Por último, las diez indicadoras que van de **A** a **SUSP** suman exactamente 1 en todas las filas, lo que demuestra que son una codificación disyuntiva de **CLASS**. Mantenerlas junto a **CLASS** supondría duplicar información y, lo que es peor, haría singular la matriz de covarianzas e impediría calcular la distancia de Mahalanobis, así que también se eliminan.

2.4 Matriz de correlaciones

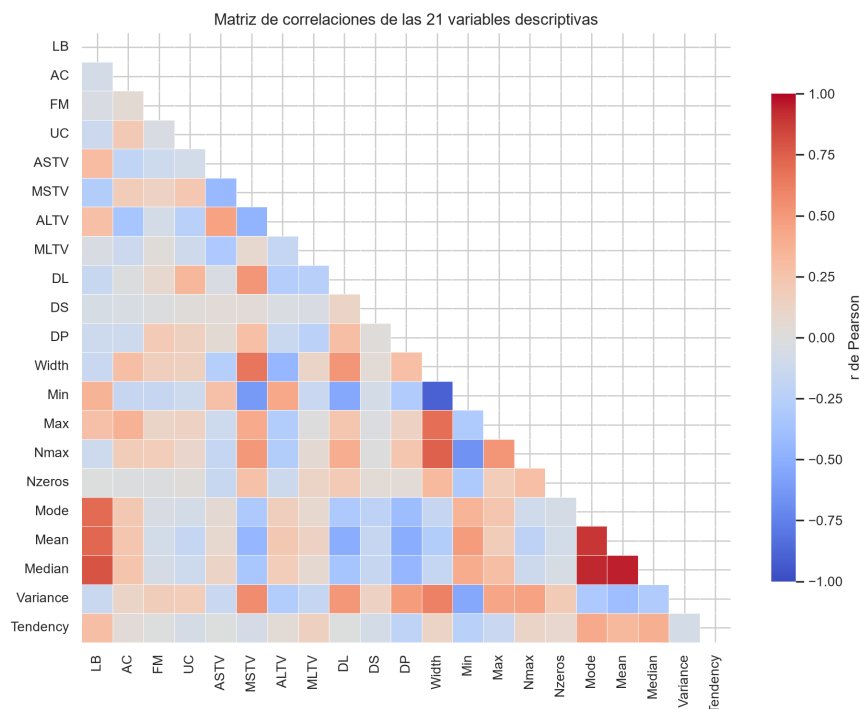
Listado 8. Celda 2.4. Matriz de correlaciones entre las variables numericas

```
# Celda 2.4. Matriz de correlaciones entre las variables numericas.
# Objetivo: cuantificar la redundancia lineal entre descriptores. La matriz se
# calcula solo sobre las 21 variables descriptivas reales, porque incluir los
# identificadores, las one-hot o las etiquetas produciria correlaciones
# espurias y una figura ilegible.
# Salidas: la figura fig_correlaciones y la lista de pares muy correlacionados.
# Variables descriptivas: se excluyen identificadores (FileName, Date, SegFile,
# b, e), la redundante LBE, la constante DR, las one-hot y las etiquetas.
VARIABLES = ["LB", "AC", "FM", "UC", "ASTV", "MSTV", "ALTV", "MLTV",
             "DL", "DS", "DP", "Width", "Min", "Max", "Nmax", "Nzeros",
             "Mode", "Mean", "Median", "Variance", "Tendency"]
# Se usan solo las filas validas; las 3 de artefacto se depuran en la seccion 3.
corr = df_bruto[VARIABLES].dropna().corr(method="pearson")
plt.figure(figsize=(9.5, 7.6))
mascara = np.triu(np.ones_like(corr, dtype=bool)) # oculta el triangulo superior
# Se representa el color sin anotar cada coeficiente: con 21 variables son
# 210 numeros que resultarian ilegibles al reducir la figura al ancho de la
# pagina. Los pares fuertes se listan a continuacion en forma de tabla.
sns.heatmap(corr, mask=mascara, annot=False, cmap="coolwarm",
            center=0, vmin=-1, vmax=1, linewidths=0.3,
            cbar_kws={"shrink": 0.8, "label": "r de Pearson"})
plt.title("Matriz de correlaciones de las 21 variables descriptivas")
plt.tight_layout()
guardar("fig_correlaciones")
# Extraccion de los pares fuertemente correlacionados. Se recorre solo el
# triangulo inferior para no repetir cada par dos veces.
pares = []
for i in range(len(VARIABLES)):
    for j in range(i + 1, len(VARIABLES)):
        r = corr.iloc[i, j]
        if abs(r) >= 0.70:
            pares.append((VARIABLES[i], VARIABLES[j], round(float(r), 3)))
pares = sorted(pares, key=lambda t: -abs(t[2]))
print("Pares de variables con |r| >= 0.70:\n")
for a, b, r in pares:
    print(" %-9s y %-9s r = %+.3f" % (a, b, r))
RES["pares_correlacionados"] = pares
RES["corr_media_abs"] = float(np.abs(corr.values[np.triu_indices_from(corr, k=1)]).mean().round(3))
print("\nCorrelacion absoluta media entre pares: %.3f" % RES["corr_media_abs"])
```

```
Pares de variables con |r| >= 0.70:
Mean      y Median      r = +0.948
Mode      y Median      r = +0.933
Width     y Min         r = -0.899
Mode      y Mean        r = +0.893
LB         y Median      r = +0.789
Width     y Nmax        r = +0.747
LB         y Mean        r = +0.723
LB         y Mode        r = +0.709
```

[Salida recortada: 1 líneas mas. La salida completa esta en el cuaderno del repositorio.]

Figura 1
Matriz de correlaciones de las 21 variables descriptivas



Nota. Coeficientes de correlación de Pearson calculados sobre las 2 126 observaciones válidas. Solo se representa el triángulo inferior para evitar la duplicación de cada par. El valor numérico de los pares más correlacionados aparece en la salida que precede a la figura. Elaboración propia.

La matriz revela una estructura de redundancia muy clara y, además, clínicamente interpretable. Existe un primer bloque formado por **Mean**, **Median**, **Mode** y **LB**, con correlaciones que van de 0,71 a 0,95, siendo el par **Mean** y **Median** el más redundante de todo el conjunto. Las cuatro variables miden lo mismo, esto es, dónde se sitúa la frecuencia cardiaca fetal, y solo se diferencian en el estimador que emplean.

Un segundo bloque agrupa a los descriptores de dispersión del histograma. La amplitud **Width** correlaciona a $-0,90$ con **Min**, a $0,75$ con **Nmax** y a $0,69$ con **Max**. La relación resulta ser, de hecho, exacta, puesto que **Width** es igual a **Max** menos **Min**, como se comprobará en la sección 4; la correlación lineal por pares no llega a revelarla porque la dependencia involucra a tres variables a la vez.

También aparecen correlaciones negativas informativas. La variable **ASTV** correlaciona a $-0,43$ con **MSTV**, y **Variance** a $-0,55$ con **Min**, lo que expresa que cuanto mayor es el porcentaje de tiempo con variabilidad anormal, menor es la variabilidad media efectiva del trazado. En conjunto, la redundancia es moderada, con una correlación absoluta media de 0,234, y está concentrada en esos dos bloques. La consecuencia práctica es que una parte apreciable de la varianza total se concentra en pocas direcciones, lo que justifica recurrir al análisis de componentes principales tanto para visualizar como para mitigar la maldición de la dimensionalidad que sufre DBSCAN.

2.5 Distribuciones de las variables clave

Listado 9. Celda 2.5. Distribucion de las variables mas relevantes

```

# Celda 2.5. Distribucion de las variables mas relevantes.
# Objetivo: visualizar la forma de la distribucion de un subconjunto
# representativo de variables, para confirmar graficamente la asimetria que
# ya se detecto numericamente en la celda 2.2.
# Salidas: las figuras fig_distribuciones y fig_boxplots.
VARS_CLAVE = ["LB", "ASTV", "MSTV", "ALTV", "MLTV", "AC",
              "UC", "DL", "Width", "Variance", "Mean", "Nmax"]
datos_validos = df_bruto[VARIABLES].dropna()
# Histogramas con estimacion de densidad
fig, ejes = plt.subplots(3, 4, figsize=(11, 6.6))
for eje, col in zip(ejes.flatten(), VARS_CLAVE):
    sns.histplot(datos_validos[col], bins=30, kde=True, ax=eje, color="#4C72B0")
    eje.set_title(col + " (asimetria = %.2f)" % datos_validos[col].skew(), fontsize=10)
    eje.set_xlabel("")
    eje.set_ylabel("")
fig.suptitle("Distribucion de las variables mas relevantes", fontsize=13)
plt.tight_layout()
guardar("fig_distribuciones")
# Diagramas de caja sobre datos estandarizados. Se estandariza solo para poder
# dibujar todas las variables en un mismo eje; el objetivo es comparar la
# CANTIDAD de puntos que quedan fuera de los bigotes.
datos_z = (datos_validos - datos_validos.mean()) / datos_validos.std()
plt.figure(figsize=(10.5, 4.2))
sns.boxplot(data=datos_z, orient="v", fliersize=1.5, color="#DD8452")
plt.xticks(rotation=90)
plt.axhline(0, color="grey", lw=0.8)
plt.ylabel("Valor estandarizado (z)")
plt.title("Diagramas de caja de las 21 variables estandarizadas")
plt.tight_layout()
guardar("fig_boxplots")
# Cuantificacion del numero de atipicos univariantes por variable segun la
# regla clasica de Tukey: fuera de [Q1 - 1.5*RIC, Q3 + 1.5*RIC].
q1 = datos_validos.quantile(0.25)
q3 = datos_validos.quantile(0.75)
ric = q3 - q1
atipicos_por_var = ((datos_validos < (q1 - 1.5 * ric)) | (datos_validos > (q3 + 1.5 * ric))).sum()
atipicos_por_var = atipicos_por_var.sort_values(ascending=False)
print("Numero de valores atipicos univariantes (criterio de Tukey) por variable:\n")
print(atipicos_por_var.to_string())
RES["atipicos univariantes"] = atipicos_por_var.head(8).to_dict()

```

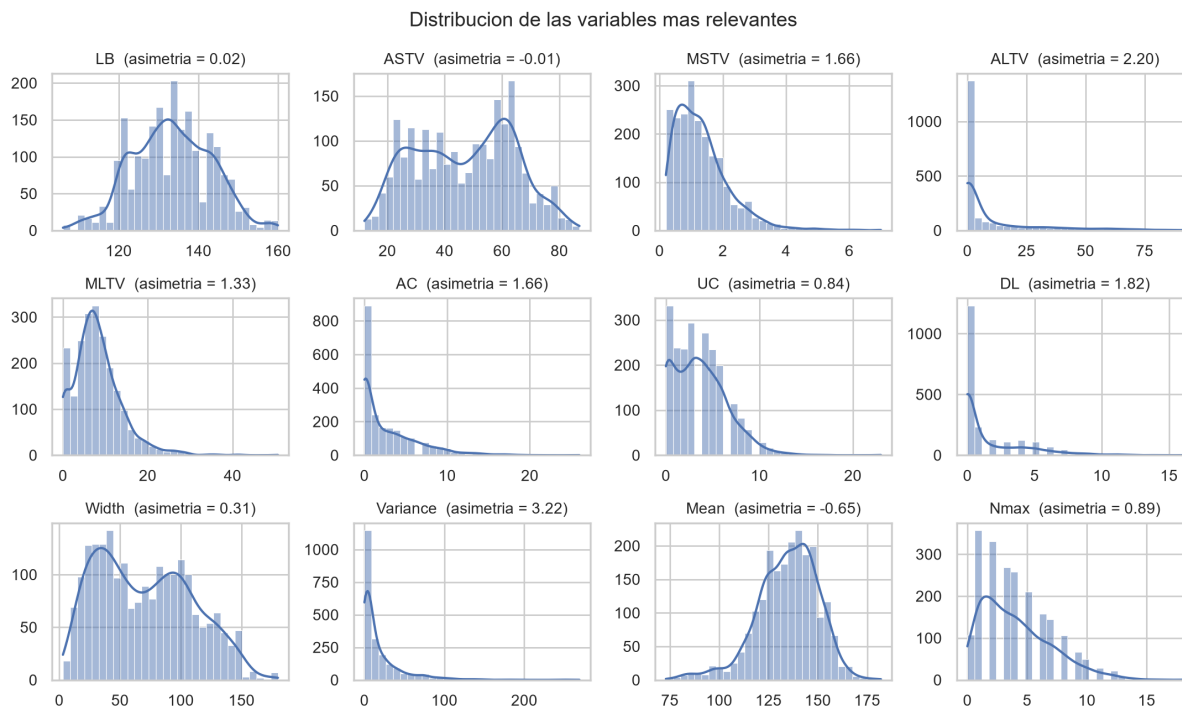
Numero de valores atipicos univariantes (criterio de Tukey) por variable:

Nzeros	502
FM	310
ALTV	309
Variance	184
DP	178
AC	83
DL	81
Mode	73
MLTV	71

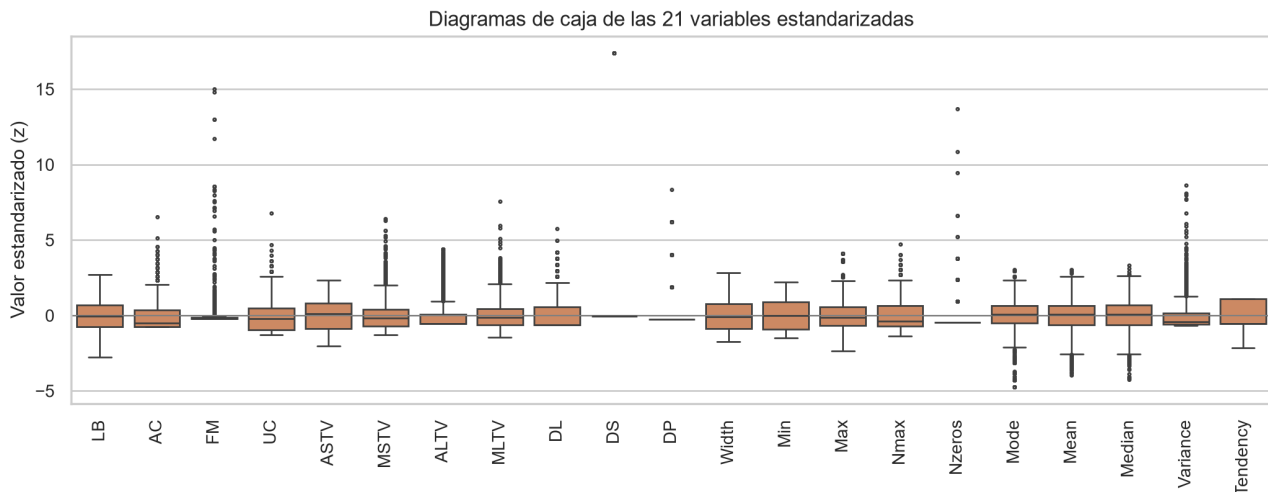
[Salida recortada: 12 líneas más. La salida completa está en el cuaderno del repositorio.]

Figura 2

Distribución de las variables más relevantes



Nota. Histogramas con estimación de densidad por núcleos. Entre paréntesis se indica el coeficiente de asimetría de cada variable. Elaboración propia.

Figura 3*Diagramas de caja de las 21 variables estandarizadas*

Nota. Las variables se estandarizaron únicamente para poder representarlas en un mismo eje. Los puntos situados fuera de los bigotes son los valores atípicos según el criterio de Tukey. Elaboración propia.

Los histogramas confirman lo que ya anticipaban la asimetría y la curtosis. Solo **LB**, **Mean**, **Mode** y **Median** son aproximadamente simétricas; el resto son distribuciones fuertemente sesgadas a la derecha, con una masa enorme de ceros en los conteos de eventos como **DS**, **DP** o **Nzeros**.

Los diagramas de caja muestran que prácticamente todas las variables tienen puntos fuera de los bigotes, lo que constituye una advertencia metodológica de primer orden. Si se aplicara una regla univariante y se eliminara toda fila con algún valor atípico, se perdería una fracción enorme del conjunto y, además, serían precisamente los casos clínicamente más interesantes los que desaparecerían. La detección de anomalías tiene que ser multivariante.

3. Tratamiento de los valores faltantes

3.1 Diagnóstico

Listado 10. Celda 3.1. Diagnostico de los valores faltantes

```
# Celda 3.1. Diagnostico de los valores faltantes.
# Objetivo: cuantificar los faltantes y, sobre todo, describir su PATRON. La
# decision de imputar o eliminar depende del patron y no del recuento:
# faltantes dispersos y faltantes concentrados en filas completas son
# problemas distintos que admiten soluciones opuestas.
# Salidas: la serie faltantes y los indices filas_con_na.
faltantes = df_bruto.isnull().sum()
faltantes_pos = faltantes[faltantes > 0].sort_values(ascending=False)
print("=" * 78)
print("DIAGNOSTICO DE VALORES FALTANTES")
print("=" * 78)
print("Celdas faltantes en total :", int(faltantes.sum()))
print("Columnas afectadas      :", int((faltantes > 0).sum()), "de", df_bruto.shape[1])
print("Porcentaje sobre el total : %.3f %%"
      % (100 * faltantes.sum() / (df_bruto.shape[0] * df_bruto.shape[1])))
print("\nFaltantes por columna (solo las afectadas):")
print(faltantes_pos.to_string())
# Analisis del patron: se cuenta cuantos faltantes tiene CADA FILA.
na_por_fila = df_bruto.isnull().sum(axis=1)
filas_con_na = df_bruto.index[na_por_fila > 0]
print("\nFilas con al menos un valor faltante:", len(filas_con_na))
print("Indices de esas filas:", list(filas_con_na))
print("\nNumero de valores faltantes en cada una de ellas (sobre 40 columnas):")
print(na_por_fila[filas_con_na].to_string())
print("\nContenido de las filas sospechosas (columnas 8 a 20):")
print(df_bruto.loc[filas_con_na].iloc[:, 8:20].to_string())
RES["faltantes_total"] = int(faltantes.sum())
RES["filas con na"] = [int(i) for i in filas_con_na]
```

```
=====
DIAGNOSTICO DE VALORES FALTANTES
=====
Celdas faltantes en total : 106
Columnas afectadas      : 40 de 40
Porcentaje sobre el total : 0.124 %
Faltantes por columna (solo las afectadas):
```

```
FileName      3
Date          3
SegFile       3
[Salida recortada: 51 líneas mas. La salida completa esta en el cuaderno del repositorio.]
```

El diagnóstico es concluyente y desmonta la lectura ingenua del problema. Hay 106 celdas faltantes repartidas en 39 columnas, pero todas ellas se concentran en solo 3 filas, las de índice 2126, 2127 y 2128, que son las tres últimas del archivo. Esas tres filas no son pacientes con datos incompletos: son artefactos de exportación de la hoja de cálculo original. La primera está completamente vacía, la segunda contiene ceros de relleno en **DL**, **DS**, **DP** y **DR**, y la tercera es una fila de totales con valores como **FM** igual a 564 o **MLTV** igual a 50,7, que están varios órdenes de magnitud fuera del rango clínico de cualquiera de esas variables. Ninguna de las tres tiene valor en **NSP** ni en **CLASS**, es decir, ningún obstetra las diagnosticó, sencillamente porque no existen como observaciones.

3.2 Decisión: eliminar frente a imputar

En términos de la tipología de Rubin (1976), este no es un caso de datos faltantes completamente al azar, ni al azar, ni no al azar. No hay ningún dato que falte; lo que hay son filas que no son observaciones. La discusión sobre si conviene la media, la mediana o la moda solo es pertinente cuando la fila representa a un individuo real del que se desconoce alguna medida, y no es este el caso.

La decisión adoptada es, por tanto, eliminar las tres filas. La justifican varios argumentos convergentes. El primero atañe a la naturaleza del dato: eliminar es correcto porque no se está descartando información clínica, mientras que imputar fabricaría tres pacientes ficticios. El segundo es el coste, que resulta despreciable, ya que se pierden 3 filas de 2 129, apenas el 0,14 % del conjunto. El tercero, y el más importante para este trabajo, es el efecto sobre los modelos: la fila de totales, de conservarse imputada, se convertiría en el valor atípico más extremo de todo el conjunto y capturaría buena parte de la capacidad de detección, además de desplazar los centroides de K-Means, que no son robustos frente a valores extremos. A ello se añade un argumento de trazabilidad, porque la imputación enmascararía un error de origen que conviene dejar documentado.

Imputar con la media sería, por añadidura, internamente incoherente, ya que se estaría usando la media de una columna que la propia fila de totales ha contaminado para completar esa misma fila.

De todo ello se deriva una regla general que vale la pena retener: antes de elegir una estrategia de imputación hay que preguntarse si la fila representa a una entidad real, y solo cuando la respuesta es afirmativa tiene sentido comparar la media, la mediana, la moda o el vecino más cercano.

Listado 11. Celda 3.2. Aplicación de la decisión: depuración del conjunto

```
# Celda 3.2. Aplicación de la decisión: depuración del conjunto.
# Objetivo: eliminar las filas de artefacto y verificar que el conjunto
#   resultante queda completo y es coherente.
# Salidas: df, el DataFrame depurado que se usa en el resto del trabajo.
n_antes = len(df_bruto)
# Criterio de eliminación: una fila sin diagnóstico NSP no es una observación
#   clínica. Es un criterio semántico, basado en el significado de la variable, y
#   resulta más seguro que un dropna() global, que dependería del azar de cada
#   columna.
df = df_bruto.dropna(subset=["NSP"]).reset_index(drop=True)
```

```

print("Filas antes de la depuracion :", n_antes)
print("Filas despues de la depuracion:", len(df))
print("Filas eliminadas      :", n_antes - len(df),
      "(%.2f %% del conjunto)" % (100 * (n_antes - len(df)) / n_antes))
# El conjunto debe quedar sin ningun faltante.
faltantes_despues = df.isnull().sum().sum()
print("\nValores faltantes tras la depuracion:", int(faltantes_despues))
assert faltantes_despues == 0, "Quedan faltantes: revisar el criterio de depuracion"
print("Verificacion superada: el conjunto esta completo.")
# Verificacion de coherencia de los rangos clinicos.
print("\nRangos de tres variables de control tras la depuracion:")
for col in ["LB", "FM", "MLTV"]:
    print("  %-6s min = %7.1f  max = %7.1f" % (col, df[col].min(), df[col].max()))
print("\nLos maximos vuelven a rangos fisiologicos, lo que confirma que la fila")
print("de totales, que aportaba FM = 564 y MLTV = 50.7, ha desaparecido.")
RES["n_filas_final"] = int(len(df))
RES["n_filas_eliminadas"] = int(n_antes - len(df))

```

```

Filas antes de la depuracion : 2129
Filas despues de la depuracion: 2126
Filas eliminadas      : 3 (0.14 % del conjunto)
Valores faltantes tras la depuracion: 0
Verificacion superada: el conjunto esta completo.
Rangos de tres variables de control tras la depuracion:
  LB   min =   106.0  max =   160.0
  FM   min =    0.0  max =   564.0
  MLTV min =    0.0  max =    50.7
[Salida recortada: 3 lineas mas. La salida completa esta en el cuaderno del repositorio.]

```

3.3 Experimento de validación

La decisión anterior se apoya en el significado de los datos. Para respaldarla también de forma cuantitativa, y para responder de manera completa a la pregunta que plantea el enunciado sobre la media, la mediana y la moda, se diseña un experimento controlado en cuatro pasos. Se parte del conjunto ya depurado, que está completo; se borra artificialmente el 10 % de los valores numéricos de forma completamente aleatoria, lo que reproduce un mecanismo de pérdida completamente al azar; se reconstruye la matriz con cuatro estrategias distintas, media, mediana, moda y vecinos más cercanos con k igual a 5; y por último se mide el error frente al valor verdadero, que en este montaje sí se conoce. Este diseño permite comparar estrategias contra una verdad de referencia, algo imposible cuando los faltantes son reales.

Listado 12. Celda 3.3. Experimento controlado de imputacion

```

# Celda 3.3. Experimento controlado de imputacion.
# Objetivo: comparar media, mediana, moda y KNN sobre faltantes SIMULADOS, donde
# el valor verdadero se conoce y el error puede medirse.
# Metricas: RMSE normalizado, que mide el error de reconstruccion y conviene que
# sea bajo, y el ratio de desviaciones tipicas, que mide la conservacion de la
# dispersion y cuyo valor ideal es 1.
# Salidas: el DataFrame comparacion_imp y la figura fig_imputacion.
# Paso 1: matriz completa de referencia
X_completa = df[VARIABLES].copy()
# Paso 2: borrado aleatorio del 10 % de las celdas
generador = np.random.default_rng(SEMILLA)
mascara_na = generador.random(X_completa.shape) < 0.10
X_perforada = X_completa.mask(mascara_na)
print("Celdas borradas artificialmente:", int(mascara_na.sum()),
      "(%.1f %% de la matriz)" % (100 * mascara_na.mean()))
# Paso 3: reconstruccion con cuatro estrategias
ESTRATEGIAS = {
    "Media": SimpleImputer(strategy="mean"),
    "Mediana": SimpleImputer(strategy="median"),
    "Moda": SimpleImputer(strategy="most_frequent"),
    "KNN (k=5)": KNNImputer(n_neighbors=5),
}
# Escala de referencia para normalizar el error: la desviacion tipica real de
# cada variable. Sin ella el RMSE quedaria dominado por Variance y Width.
sigma = X_completa.std().replace(0, np.nan)
filas_resultado = []
imputaciones = {}
for nombre, imputador in ESTRATEGIAS.items():
    # fit_transform aprende el estadistico de cada columna y rellena los huecos.
    X_rec = pd.DataFrame(imputador.fit_transform(X_perforada),
                        columns=VARIABLES, index=X_completa.index)
    imputaciones[nombre] = X_rec
    # El error se mide solo en las celdas borradas; las demas son exactas.
    error = (X_rec - X_completa)[mascara_na]
    rmse_norm = float(np.sqrt(((error / sigma) ** 2).stack().mean()))
    # Conservacion de la dispersion: toda imputacion por una constante central
    # reduce artificialmente la varianza de la variable.
    ratio_sd = float((X_rec.std() / X_completa.std()).mean())
    # Conservacion de la estructura de correlaciones, importante para el PCA.
    corr_real = X_completa.corr().values[np.triu_indices(len(VARIABLES), k=1)]
    corr_rec = X_rec.corr().values[np.triu_indices(len(VARIABLES), k=1)]
    error_corr = float(np.abs(corr_real - corr_rec).mean())
    filas_resultado.append({
        "Estrategia": nombre,
        "RMSE normalizado": round(rmse_norm, 4),
        "Ratio desv. tipica": round(ratio_sd, 4),
    })

```

```

    "Error medio de correlacion": round(error_corr, 4),
})
comparacion_imp = pd.DataFrame(filas_resultado).set_index("Estrategia")
print("\nComparacion de estrategias de imputacion (faltantes simulados al 10 %):\n")
print(comparacion_imp.to_string())
RES["comparacion_imputacion"] = comparacion_imp.reset_index().to_dict("records")

```

Celdas borradas artificialmente: 4420 (9.9 % de la matriz)
 Comparacion de estrategias de imputacion (faltantes simulados al 10 %):

Estrategia	RMSE normalizado	Ratio desv. tipica	Error medio de correlacion
Media	0.3815	0.9491	0.0266
Mediana	0.3986	0.9530	0.0280
Moda	0.4814	0.9749	0.0378
KNN (k=5)	0.2242	0.9834	0.0064

Listado 13. Celda 3.4. Efecto visual de la imputacion sobre las distribuciones

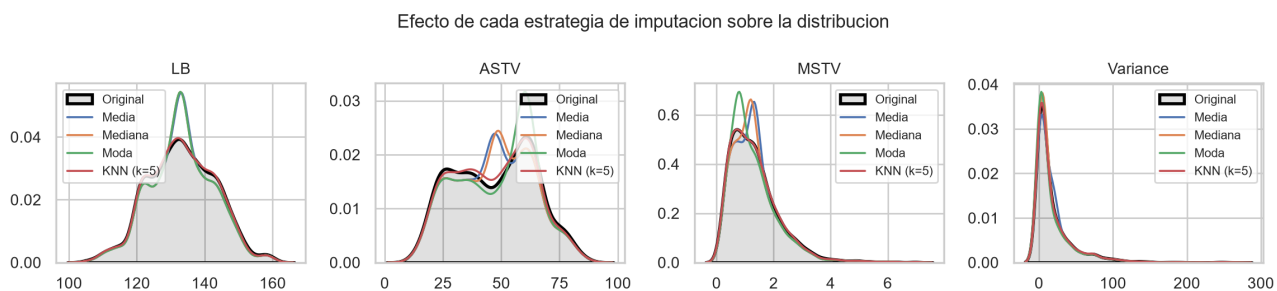
```

# Celda 3.4. Efecto visual de la imputacion sobre las distribuciones.
# Objetivo: mostrar graficamente la distorsion que introduce cada estrategia. La
# comparacion se hace sobre cuatro variables de forma distinta: una simetrica,
# una sesgada, una porcentual y una de conteo.
# Salidas: la figura fig_imputacion.
def comparar_distribuciones(df_real, dict_imputados, columnas, ncols=4):
    """Superpone la distribucion real y la imputada para cada variable.
    Parametros
    -----
    df_real : DataFrame
        Datos completos originales, que hacen de verdad de referencia.
    dict_imputados : dict
        Diccionario {nombre_estrategia: DataFrame imputado}.
    columnas : list
        Variables a representar.
    ncols : int
        Numero de columnas de la rejilla de subgraficos.
    """
    nrows = int(np.ceil(len(columnas) / ncols))
    fig, ejes = plt.subplots(nrows, ncols, figsize=(ncols * 2.9, nrows * 2.7))
    ejes = np.atleast_1d(ejes).flatten()
    for eje, col in zip(ejes, columnas):
        # Distribucion verdadera como referencia, con area rellena.
        sns.kdeplot(df_real[col], ax=eje, color="black", lw=2,
                    label="Original", fill=True, alpha=0.12)
        # Una curva por estrategia de imputacion.
        for nombre, X_rec in dict_imputados.items():
            sns.kdeplot(X_rec[col], ax=eje, lw=1.3, label=nombre)
        eje.set_title(col, fontsize=10)
        eje.set_xlabel("")
        eje.set_ylabel("")
        eje.legend(fontsize=8)
    # Se eliminan los ejes sobrantes de la rejilla.
    for eje in ejes[len(columnas):]:
        fig.delaxes(eje)
    fig.suptitle("Efecto de cada estrategia de imputacion sobre la distribucion",
                fontsize=12)
    plt.tight_layout()
    comparar_distribuciones(X_completa, imputaciones,
                            ["LB", "ASTV", "MSTV", "Variance"])
    guardar("fig_imputacion")

```

Figura 4

Efecto de cada estrategia de imputación sobre la distribución



Nota. Curvas de densidad obtenidas tras borrar de forma aleatoria el 10 % de los valores y reconstruirlos con cuatro estrategias distintas. El área sombreada corresponde a la distribución original. Elaboración propia.

El experimento cuantifica lo que la teoría anticipaba. La imputación por vecinos más cercanos gana en las tres métricas a la vez: su RMSE normalizado es de 0,224, frente a 0,382 de la media, 0,399 de la mediana y 0,481 de la moda, de modo que reduce el error de reconstrucción casi a la mitad. La razón es que estima cada hueco a partir de las cinco observaciones más parecidas y aprovecha así la correlación entre variables documentada en el apartado 2.4, estructura que los tres imputadores por constante ignoran por completo.

Todas las estrategias comprimen la dispersión, puesto que el ratio de desviaciones típicas cae por debajo de 1 en los cuatro casos. Sustituir el 10 % de los valores por un único número central reduce de forma mecánica la varianza. La media es la que más la degrada, con un ratio de 0,949, y el método de vecinos la que menos, con 0,983. Este último es además el único que preserva la estructura de correlaciones, algo decisivo para el análisis de componentes principales posterior: su error medio en la matriz de correlaciones es de 0,006, cuatro veces menor que el de la media, que llega a 0,027, y seis veces menor que el de la moda, que alcanza 0,038.

La figura muestra un aspecto que el RMSE no llega a capturar. La media, dibujada en azul, crea un pico artificial justo en el valor medio de **ASTV** y de **MSTV**, una moda que no existe en la distribución real. La moda, en verde, resulta aún peor, porque concentra masa en el valor más frecuente y deforma por completo la distribución de **LB** y de **ASTV**. La curva del método de vecinos, en rojo, se superpone casi exactamente a la distribución original.

En conclusión, si en este conjunto hubiera habido faltantes reales y dispersos, la elección correcta habría sido la imputación por vecinos más cercanos, y por un margen amplio. Entre las estrategias simples, la media y la mediana quedan muy igualadas, ya que la primera reconstruye algo mejor y la segunda conserva algo mejor la dispersión, mientras que la moda debe descartarse para variables continuas. Pero como los faltantes de este conjunto proceden de tres filas que no son observaciones, la decisión que finalmente se aplica es eliminarlas, y el conjunto de trabajo queda con 2 126 registros completos.

4. Preparación de la matriz de características

Antes de aplicar los algoritmos hay que decidir con qué columnas se los alimenta, y es probablemente la decisión de mayor impacto de todo el trabajo. Se toma a partir de las evidencias reunidas en el análisis exploratorio.

Se descartan **FileName**, **Date**, **SegFile**, **b** y **e** porque identifican el registro y no describen al feto, según se vio en el apartado 2.3. Se descarta **LBE** por ser un duplicado exacto de **LB**, y **DR** por ser constante e igual a cero, de manera que su varianza es nula y no aporta información alguna. Se descarta **Width** porque es una combinación lineal exacta de otras dos columnas. Se descartan las diez indicadoras que van de **A** a **SUSP** por tratarse de la codificación disyuntiva de la etiqueta **CLASS**. Y se apartan **CLASS** y **NSP**, que quedan reservadas como verdad de referencia externa. Las veinte columnas restantes son descriptores cuantitativos genuinos del trazado y constituyen la matriz de trabajo.

4.1 Por qué importa eliminar la variable Width

La igualdad entre **Width** y la diferencia de **Max** y **Min** se cumple en las 2 126 filas sin una sola excepción. Eso hace que la matriz de datos tenga rango 20 con 21 columnas, es decir, que su matriz de covarianzas sea singular y no pueda invertirse. La distancia de Mahalanobis exige exactamente esa inversión, de modo que mantener **Width** haría fracasar, o al menos volvería numéricamente inestable, uno de los cinco detectores de la sección 5. Es un ejemplo claro de

por qué el análisis exploratorio debe preceder al modelado.

4.2 Por qué estandarizar

K-Means, DBSCAN, el factor local de atipicidad y la distancia de Mahalanobis se basan todos ellos en distancias. Con las escalas originales, **Variance**, cuyo rango va de 0 a 269, pesaría cientos de veces más que **MSTV**, que se mueve entre 0 y 7, y lo haría únicamente por su unidad de medida. La estandarización, que transforma cada valor restándole la media de su columna y dividiéndolo entre la desviación típica, deja todas las variables con media cero y desviación uno, de manera que cada descriptor contribuye en igualdad de condiciones.

Listado 14. Celda 4.1. Construcción de la matriz de características

```
# Celda 4.1. Construcción de la matriz de características.
# Objetivo: materializar las decisiones descritas arriba y verificar
# numericamente que se resuelven los problemas detectados.
# Salidas: X, la matriz sin escalar; Z, la matriz estandarizada; y las etiquetas
# y_nsp e y_class, apartadas para la validación externa.
# Verificación previa de las redundancias detectadas en el EDA.
print("Comprobaciones de redundancia:")
print(" LBE idéntica a LB      :", bool((df["LBE"] == df["LB"]).all()))
print(" DR constante         :", df["DR"].nunique() == 1,
      "(valores únicos:", df["DR"].unique().tolist(), ")")
print(" Width == Max - Min    :", bool((df["Width"] == (df["Max"] - df["Min"])).all()))
# Conjunto final de variables
VARIABLES_MODELO = ["LB", "AC", "FM", "UC", "ASTV", "MSTV", "ALTV", "MLTV",
                   "DL", "DS", "DP", "Min", "Max", "Nmax", "Nzeros",
                   "Mode", "Mean", "Median", "Variance", "Tendency"]
X = df[VARIABLES_MODELO].copy() # matriz en unidades originales
y_nsp = df["NSP"].astype(int).values # etiqueta apartada (3 niveles)
y_class = df["CLASS"].astype(int).values # etiqueta apartada (10 patrones)
print("\nMatriz de características:", X.shape[0], "observaciones x",
      X.shape[1], "variables")
# Estandarización: StandardScaler calcula mu y sigma por columna y aplica
# z = (x - mu) / sigma.
escalador = StandardScaler()
Z = escalador.fit_transform(X)
print("\nEfecto de la estandarización (3 variables de control):")
print(" %10s %12s %12s | %10s %10s" % ("Variable", "media orig.", "dev. orig.",
                                       "media z", "dev. z"))

for v in ["LB", "MSTV", "Variance"]:
    k = VARIABLES_MODELO.index(v)
    print(" %10s %12.2f %12.2f | %10.2f %10.2f"
          % (v, X[v].mean(), X[v].std(), Z[:, k].mean(), Z[:, k].std()))
# La matriz debe tener rango completo para que su covarianza sea invertible.
rango = np.linalg.matrix_rank(Z)
print("\nRango de la matriz estandarizada:", rango, "de", Z.shape[1], "columnas")
assert rango == Z.shape[1], "La matriz sigue siendo singular: revisar redundancias"
print("Verificación superada: la covarianza es invertible.")
RES["n_variables_modelo"] = len(VARIABLES_MODELO)
RES["variables_modelo"] = VARIABLES_MODELO
```

```
Comprobaciones de redundancia:
LBE idéntica a LB      : True
DR constante          : True (valores únicos: [0.0] )
Width == Max - Min    : True
Matriz de características: 2126 observaciones x 20 variables
Efecto de la estandarización (3 variables de control):
Variable  media orig.  dev. orig. |  media z   dev. z
LB         133.30      9.84 |    0.00     1.00
MSTV        1.33       0.88 |    0.00     1.00
[Salida recortada: 4 líneas más. La salida completa está en el cuaderno del repositorio.]
```

Listado 15. Celda 4.2. Análisis de componentes principales

```
# Celda 4.2. Análisis de componentes principales.
# Objetivo: cuantificar la dimensionalidad efectiva del problema y obtener una
# proyección en dos dimensiones que se reutilizara para visualizar tanto las
# anomalías de la sección 5 como los grupos de la sección 6.
# Nota: el PCA se ajusta sobre datos ya estandarizados, lo que equivale a
# diagonalizar la matriz de correlaciones y no la de covarianzas.
# Salidas: el objeto pca, la matriz Z_pca con todas las componentes y Z2 con las
# dos primeras.
pca = PCA(random_state=SEMILLA)
Z_pca = pca.fit_transform(Z)
varianza = pca.explained_variance_ratio_
acumulada = np.cumsum(varianza)
print("Varianza explicada por las primeras componentes principales:\n")
print(" CP    individual    acumulada")
for i in range(8):
    print(" %2d %8.1f %% %8.1f %%" % (i + 1, 100 * varianza[i], 100 * acumulada[i]))
# Componentes necesarias para retener el 80 % y el 90 % de la varianza.
n80 = int(np.searchsorted(acumulada, 0.80) + 1)
n90 = int(np.searchsorted(acumulada, 0.90) + 1)
print("\nComponentes necesarias para el 80 % de varianza:", n80)
print("Componentes necesarias para el 90 % de varianza:", n90)
# Gráfico de sedimentación y varianza acumulada
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 3.5))
a1.bar(range(1, len(varianza) + 1), 100 * varianza, color="#4C72B0")
a1.set_xlabel("Componente principal")
a1.set_ylabel("Varianza explicada (%)")
a1.set_title("Gráfico de sedimentación")
a2.plot(range(1, len(acumulada) + 1), 100 * acumulada, "o-", color="#C44E52")
```



```

a2.axhline(80, ls="--", color="grey", lw=1)
a2.axhline(90, ls=":", color="grey", lw=1)
a2.set_xlabel("Numero de componentes")
a2.set_ylabel("Varianza acumulada (%)")
a2.set_title("Varianza acumulada")
plt.tight_layout()
guardar("fig_pca_varianza")
# Las cargas indican con que peso entra cada variable original en cada
# componente, y son la clave para interpretarlas.
cargas = pd.DataFrame(pca.components_[1:2].T, index=VARIABLES_MODELO,
                      columns=["CP1", "CP2"]).round(3)
print("\nVariables con mayor carga en CP1 (negativas y positivas):")
print(cargas["CP1"].sort_values().head(4).to_string())
print(cargas["CP1"].sort_values().tail(4).to_string())
print("\nVariables con mayor carga en CP2:")
print(cargas["CP2"].sort_values().head(3).to_string())
print(cargas["CP2"].sort_values().tail(3).to_string())
Z2 = Z_pca[:, :2] # proyeccion 2D reutilizada en todas las figuras
Z_pca80 = Z_pca[:, :n80] # espacio reducido al 80 % de varianza
RES["pca_var"] = [round(float(v), 4) for v in varianza[:8]]
RES["pca_n80"] = n80

```

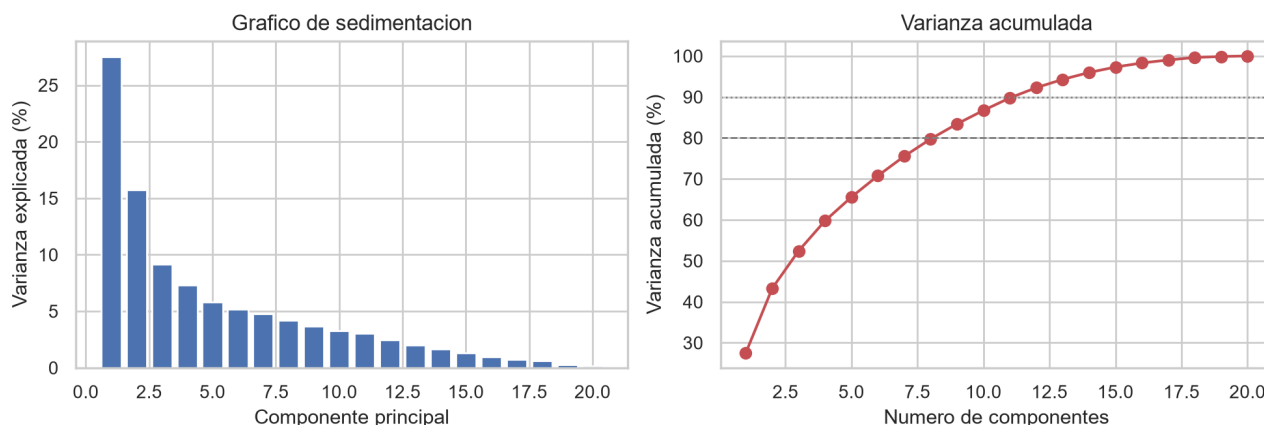
Varianza explicada por las primeras componentes principales:

CP	individual	acumulada
1	27.5 %	27.5 %
2	15.8 %	43.3 %
3	9.1 %	52.4 %
4	7.3 %	59.8 %
5	5.8 %	65.6 %
6	5.2 %	70.8 %
7	4.8 %	75.5 %
8	4.2 %	79.7 %

[Salida recortada: 21 líneas mas. La salida completa esta en el cuaderno del repositorio.]

Figura 5

Sedimentación y varianza acumulada del análisis de componentes principales



Nota. Las líneas discontinuas señalan los umbrales del 80 % y del 90 % de varianza explicada. Elaboración propia.

La primera componente explica el 27,5 % de la varianza y la segunda el 15,8 %, de modo que entre ambas apenas superan el 43 %, y hacen falta nueve componentes para alcanzar el 80 %. El problema no es, por tanto, de dimensionalidad trivial: no puede resumirse en dos ejes sin perder información sustancial. Las dos primeras componentes se emplearán únicamente para visualizar, nunca como entrada única de un modelo.

Las cargas admiten una lectura clínica directa. La primera componente contrapone las variables de tendencia central y anchura del histograma, **Mean**, **Mode**, **Median** y **Max**, frente a las de variabilidad anormal, **ASTV** y **ALTV**. La segunda separa la actividad del registro, medida por **AC**, **FM** y **UC**, de las deceleraciones. Son, en esencia, los dos ejes con los que un obstetra describiría un trazado.

5. Detección de anomalías

5.1 Planteamiento

Una anomalía, o valor atípico, es una observación que se aparta del comportamiento mayoritario del conjunto. En este problema la pregunta operativa es si resulta posible señalar automáticamente los trazados fetales inusuales sin recurrir al diagnóstico del obstetra.

Para responderla se aplican cinco técnicas pertenecientes a tres familias distintas. El Z-score y la regla de Tukey, también llamada criterio del rango intercuartílico, son métodos estadísticos univariantes: el primero marca una observación cuando alguno de sus valores tipificados supera 3 en valor absoluto y asume por tanto la normalidad de cada variable; el segundo la marca cuando algún valor cae fuera del intervalo delimitado por el primer cuartil menos una vez y media el rango intercuartílico y el tercer cuartil más esa misma cantidad, y no asume ninguna distribución, aunque sigue examinando las variables de una en una. La distancia de Mahalanobis con estimación robusta de la covarianza es un método estadístico multivariante que mide la separación al centro de la nube ponderándola por la correlación entre variables, y presupone normalidad multivariante. Isolation Forest es un ensamble de árboles que se apoya en la idea de que los puntos raros se aíslan con muy pocos cortes aleatorios, y no asume ninguna distribución. Por último, el factor local de atipicidad, conocido como LOF, compara la densidad de cada punto con la de sus vecinos y tampoco impone supuestos distribucionales.

Para que la comparación sea justa, los métodos multivariantes se calibran de manera que señalen la misma proporción de observaciones, el 5 %. La evaluación no se limita a contar cuántas marcan: se recurre a la etiqueta **NSP**, que ningún modelo ha visto, para medir el enriquecimiento en casos patológicos, magnitud que en minería de datos se conoce como lift y que se define como el cociente entre la probabilidad de que una observación sea patológica dado que el detector la ha marcado y la probabilidad de que lo sea sin más información. Un lift igual a 1 significa que el detector no aporta nada frente a elegir al azar.

5.2 Métodos univariantes de referencia

Listado 16. Celda 5.1. Metodos univariantes: Z-score y regla de Tukey

```
# Celda 5.1. Metodos univariantes: Z-score y regla de Tukey.
# Objetivo: establecer una línea base con las dos reglas clásicas. Se aplican
# variable a variable y se marca la fila cuando alguna de sus 20 variables
# resulta atípica.
# Salidas: las mascararas booleanas anom_zscore y anom_iqr.
# Z-score: se marca cuando |x - mu| / sigma supera 3. Bajo normalidad, ese
# umbral deja fuera solo el 0,27 % de los casos por variable.
puntuaciones_z = np.abs((X - X.mean()) / X.std())
anom_zscore = (puntuaciones_z > 3).any(axis=1).values
# Regla de Tukey: fuera de [Q1 - 1.5*RIC, Q3 + 1.5*RIC].
Q1 = X.quantile(0.25)
Q3 = X.quantile(0.75)
RIC = Q3 - Q1
limite_inf = Q1 - 1.5 * RIC
limite_sup = Q3 + 1.5 * RIC
anom_iqr = ((X < limite_inf) | (X > limite_sup)).any(axis=1).values
print("=" * 78)
print("METODOS UNIVARIANTES DE REFERENCIA")
print("=" * 78)
print("Z-score (|z| > 3) : %4d filas marcadas (%.1f %% del conjunto)"
      % (anom_zscore.sum(), 100 * anom_zscore.mean()))
print("Tukey (1.5 x RIC) : %4d filas marcadas (%.1f %% del conjunto)"
      % (anom_iqr.sum(), 100 * anom_iqr.mean()))
# El problema de fondo es que la probabilidad de falso positivo se acumula con
# el numero de variables analizadas simultaneamente.
p_falso_positivo = 1 - (1 - 0.0027) ** len(VARIABLES_MODELO)
print("\nProbabilidad teorica de que una fila NORMAL sea marcada por el Z-score")
print("al examinar %d variables independientes: %.1f %%"
      % (len(VARIABLES_MODELO), 100 * p_falso_positivo))
RES["n_zscore"] = int(anom_zscore.sum())
RES["n_iqr"] = int(anom_iqr.sum())
```

```
=====
METODOS UNIVARIANTES DE REFERENCIA
=====
Z-score (|z| > 3) : 343 filas marcadas (16.1 % del conjunto)
Tukey (1.5 x RIC) : 1220 filas marcadas (57.4 % del conjunto)
```

Las reglas univariantes fracasan por completo en este conjunto. El Z-score marca 343 filas, el 16,1 % del total, más de tres veces la contaminación que cabría considerar razonable. La regla de Tukey marca 1 220 filas, es decir, el 57,4 % del conjunto; un detector de anomalías que declara anómala a más de la mitad de la población sencillamente no detecta nada.

La causa es doble. Por un lado está el problema de las comparaciones múltiples: aun suponiendo normalidad e independencia, examinar 20 variables con un umbral de 3 desviaciones típicas eleva la probabilidad acumulada de falso positivo hasta el 5,3 % por fila. Por otro, y más importante, las variables no son normales, ya que el análisis exploratorio mostró asimetrías extremas y conteos con exceso de ceros; en esa situación el rango intercuartílico se vuelve minúsculo y casi cualquier valor no nulo queda fuera de los bigotes.

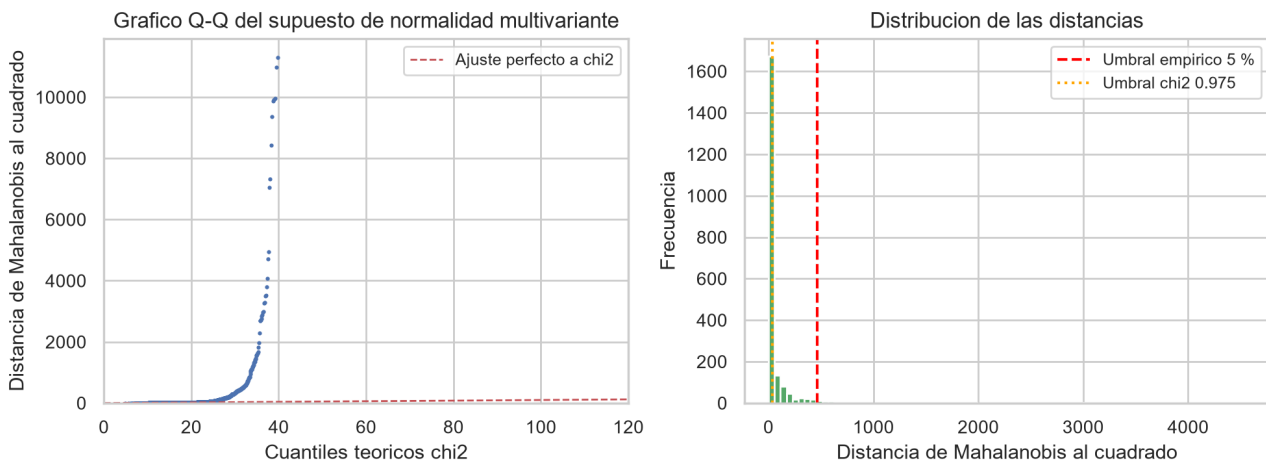
La lección es clara: la anormalidad de un trazado no reside en ninguna variable aislada, sino en la combinación de sus valores, y para capturarla hacen falta métodos multivariantes.

5.3 Distancia de Mahalanobis robusta

Listado 17. Celda 5.2. Distancia de Mahalanobis con estimacion robusta de la covarianza

```
# Celda 5.2. Distancia de Mahalanobis con estimacion robusta de la covarianza.
# Objetivo: primer metodo multivariante. La distancia de Mahalanobis mide la
# separacion al centro de la nube teniendo en cuenta la correlacion, mediante
#  $d^2(x) = (x - \mu)^T S^{-1} (x - \mu)$ . Se usa el estimador MCD, que busca el
# subconjunto mas compacto de los datos, en lugar de la media y la covarianza
# muestrales, porque estas son ellas mismas sensibles a los atipicos y
# producen el llamado efecto de enmascaramiento.
# Salidas: dist_mahalanobis y la mascara anom_mahalanobis.
# support_fraction = 0.85 indica que el MCD busca el subconjunto del 85 % de los
# datos cuya covarianza tiene el menor determinante, es decir, el nucleo denso.
mcd = MinCovDet(support_fraction=0.85, random_state=SEMILLA).fit(Z)
dist_mahalanobis = mcd.mahalanobis(Z) # devuelve  $d^2$ , no d
# Criterio A: umbral teorico chi-cuadrado. Bajo normalidad multivariante,  $d^2$ 
# sigue una distribucion chi2 con p grados de libertad.
umbral_chi2 = stats.chi2.ppf(0.975, df=len(VARIABLES_MODELO))
marcados_chi2 = dist_mahalanobis > umbral_chi2
# Criterio B: proporcion fija del 5 %, para poder comparar con los demas.
N_ANOMALIAS = int(round(0.05 * len(Z)))
umbral_5pct = np.sort(dist_mahalanobis)[-N_ANOMALIAS]
anom_mahalanobis = dist_mahalanobis >= umbral_5pct
print("Umbral teorico chi2(0.975, %d gl) = %.1f" % (len(VARIABLES_MODELO), umbral_chi2))
print(" marca %d filas (%.1f %% del conjunto)"
      % (marcados_chi2.sum(), 100 * marcados_chi2.mean()))
print("\nUmbral empirico al 5 % ( $d^2 \geq$  %.1f)" % umbral_5pct)
print(" marca %d filas (%.1f %% del conjunto)"
      % (anom_mahalanobis.sum(), 100 * anom_mahalanobis.mean()))
# Contraste del supuesto de normalidad multivariante. Si se cumpliera, los
# cuantiles empiricos de  $d^2$  coincidirian con los de la chi2.
cuantiles_teoricos = stats.chi2.ppf(
    (np.arange(len(Z)) + 0.5) / len(Z), df=len(VARIABLES_MODELO))
plt.figure(figsize=(10, 3.8))
plt.subplot(1, 2, 1)
plt.plot(cuantiles_teoricos, np.sort(dist_mahalanobis), ".", ms=2.5, color="#4C72B0")
lim = max(cuantiles_teoricos.max(), 120)
plt.plot([0, lim], [0, lim], "r--", lw=1, label="Ajuste perfecto a chi2")
plt.xlim(0, lim); plt.ylim(0, np.percentile(dist_mahalanobis, 99.5))
plt.xlabel("Cuantiles teoricos chi2"); plt.ylabel("Distancia de Mahalanobis al cuadrado")
plt.title("Grafico Q-Q del supuesto de normalidad multivariante")
plt.legend(fontsize=9)
plt.subplot(1, 2, 2)
plt.hist(dist_mahalanobis, bins=80, color="#55A868", range=(0, np.percentile(dist_mahalanobis, 99)))
plt.axvline(umbral_5pct, color="red", ls="--", label="Umbral empirico 5 %")
plt.axvline(umbral_chi2, color="orange", ls=":", label="Umbral chi2 0.975")
plt.xlabel("Distancia de Mahalanobis al cuadrado"); plt.ylabel("Frecuencia")
plt.title("Distribucion de las distancias")
plt.legend(fontsize=9)
plt.tight_layout()
guardar("fig_mahalanobis")
RES["n mahalanobis chi2"] = int(marcados_chi2.sum())
```

Umbral teorico chi2(0.975, 20 gl) = 34.2
marcaria 646 filas (30.4 % del conjunto)
Umbral empirico al 5 % ($d^2 \geq$ 465.5)
marca 106 filas (5.0 % del conjunto)

Figura 6*Contraste del supuesto de normalidad multivariante*

Nota. A la izquierda, gráfico cuantil-cuantil de la distancia de Mahalanobis robusta frente a la distribución chi-cuadrado con 20 grados de libertad. A la derecha, distribución empírica de esas distancias con los dos umbrales considerados. Elaboración propia.

El gráfico cuantil-cuantil se separa muy pronto de la diagonal, porque las distancias observadas crecen mucho más rápido que las que predice la chi-cuadrado. El supuesto de normalidad multivariante no se cumple, y esa es la razón de que el umbral teórico marque 646 observaciones, el 30,4 % del conjunto, una cifra sin ningún sentido operativo, ya que ningún servicio de obstetricia revisaría a mano tres de cada diez registros. Se conserva, por tanto, el umbral empírico del 5 %, que además permite comparar este método con Isolation Forest y con LOF en igualdad de condiciones.

5.4 Isolation Forest

El método fue propuesto por Liu, Ting y Zhou (2008) y parte de una observación sencilla. Si se particiona el espacio eligiendo al azar una variable y un punto de corte, un punto anómalo queda aislado en muy pocos cortes, mientras que uno situado en el núcleo denso de la nube necesita muchos. El algoritmo construye un bosque de árboles de partición aleatoria y asigna a cada observación una puntuación basada en su profundidad media de aislamiento.

A diferencia de casi todos los demás métodos, no modela la normalidad para buscar después desviaciones, sino que modela directamente la rareza. No asume ninguna distribución, tolera bien la alta dimensionalidad y su coste computacional es lineal en el número de observaciones.

Listado 18. Celda 5.3. Isolation Forest

```
# Celda 5.3. Isolation Forest.
# Objetivo: detectar anomalías mediante aislamiento aleatorio recursivo.
# Parametros: n_estimators = 200 fija el numero de arboles, y cuantos mas haya
# mas estable es la puntuacion; contamination = 0.05 declara la proporción
# esperada de anomalías y con ello fija el umbral de decision; max_samples =
# 256 es la submuestra por arbol recomendada por los autores, ya que
# submuestrear mejora la detección al reducir el enmascaramiento entre
# anomalías proximas.
# Salidas: score_iforest, donde un valor mayor indica mas anómalo, y la mascara
# anom_iforest.
iforest = IsolationForest(
    n_estimators=200,
    contamination=0.05,
    max_samples=256,
    random_state=SEMILLA,
    n_jobs=-1,
)
# fit_predict devuelve -1 para las anomalías y +1 para las observaciones normales.
etiquetas_if = iforest.fit_predict(Z)
anom_iforest = etiquetas_if == -1
# score_samples devuelve valores negativos y cuanto menor es el valor, mas
# anómala es la observación. Se invierte el signo para que "mayor" signifique
# "mas anómalo" y la lectura resulte natural.
```

```

score_iforest = iforest.score_samples(Z)
print("Isolation Forest")
print("  Arboles del ensamble      :", iforest.n_estimators)
print("  Observaciones marcadas    : %d (%.1f %)"
      % (anom_iforest.sum(), 100 * anom_iforest.mean()))
print("  Umbral de decision aprendido: %.4f" % (-iforest.offset_))
print("\n Puntuacion de anomalidad: min = %.3f | mediana = %.3f | max = %.3f"
      % (score_iforest.min(), np.median(score_iforest), score_iforest.max()))
# Para saber QUE hace anomalas a las observaciones marcadas se compara su perfil
# medio, en unidades z, con el del resto del conjunto.
perfil = pd.DataFrame({
    "Media z (anomalias)": Z[anom_iforest].mean(axis=0),
    "Media z (resto)": Z[~anom_iforest].mean(axis=0),
}, index=VARIABLES_MODELO)
perfil["Diferencia"] = (perfil["Media z (anomalias)"] - perfil["Media z (resto)"]).round(2)
perfil = perfil.round(2).reindex(perfil["Diferencia"].abs().sort_values(ascending=False).index)
print("\nPerfil de las anomalias detectadas (en desviaciones típicas):\n")
print(perfil.head(8).to_string())
RES["perfil iforest"] = perfil.head(8)["Diferencia"].to_dict()

```

```

Isolation Forest
  Arboles del ensamble      : 200
  Observaciones marcadas    : 107 (5.0 %)
  Umbral de decision aprendido: 0.5242
  Puntuacion de anomalidad: min = 0.365 | mediana = 0.420 | max = 0.627
Perfil de las anomalias detectadas (en desviaciones típicas):
  Media z (anomalias)  Media z (resto)  Diferencia
Variance              2.22            -0.12         2.33
[Salida recortada: 7 líneas mas. La salida completa esta en el cuaderno del repositorio.]

```

El perfil de las observaciones marcadas resulta clínicamente coherente y sirve como validación cualitativa del método. Las anomalías detectadas se sitúan muy por encima de la media general en **Variance**, con 2,33 desviaciones típicas de diferencia, en **DP**, que recoge las deceleraciones prolongadas, con 2,22, en **MSTV**, indicador de variabilidad errática a corto plazo, con 1,68, y también en **Max**, **Nmax**, **DL** y **DS**. Al mismo tiempo se sitúan muy por debajo en **Mean**, con 1,48 desviaciones típicas menos, y de forma análoga en **Mode**, **Median** y **Min**.

Traducido al lenguaje clínico, el algoritmo ha aislado trazados con bradicardia, puesto que toda la tendencia central del histograma aparece desplazada a la baja, acompañada de deceleraciones prolongadas y severas y de una variabilidad amplia y errática. Es la descripción de manual de un trazado no tranquilizador, y lo notable es que la haya reconstruido sin haber visto un solo diagnóstico.

Merece la pena señalar un matiz. La variable **ALTV** aparece por debajo de la media en las anomalías, con 0,54 desviaciones típicas menos. Isolation Forest no está capturando el fenotipo del trazado plano, caracterizado por una variabilidad reducida y monótona, sino el fenotipo errático y decelerativo. Son dos formas distintas de compromiso fetal y el detector se especializa en la segunda.

5.5 Factor local de atipicidad

El método fue propuesto por Breunig y sus colaboradores en el año 2000 y no busca puntos lejanos del centro, sino puntos situados en zonas menos densas que su propio vecindario. Para cada observación compara su densidad local con la densidad local media de sus k vecinos más próximos, de modo que un cociente muy superior a 1 indica que el punto está más solo que quienes lo rodean.

Su ventaja teórica es la capacidad de detectar anomalías locales, es decir, puntos que resultan anómalos respecto de su grupo aunque no sean extremos en el conjunto global. Su debilidad es que depende críticamente del valor de k y que la noción de densidad se degrada en dimensión alta.

Listado 19. Celda 5.4. Factor local de atipicidad (LOF)

```
# Celda 5.4. Factor local de atipicidad (LOF).
```

```
# Objetivo: detectar anomalías por contraste de densidad local.
# Parametros: n_neighbors = 20 fija el tamaño del vecindario, valor por defecto
# recomendado, ya que un vecindario demasiado pequeño introduce ruido y uno
# demasiado grande borra el carácter local del método; contamination = 0.05
# mantiene la misma proporción que Isolation Forest para poder comparar.
# Salidas: score_lof y la máscara anom_lof.
lof = LocalOutlierFactor(n_neighbors=20, contamination=0.05, n_jobs=-1)
etiquetas_lof = lof.fit_predict(Z)
anom_lof = etiquetas_lof == -1
# negative_outlier_factor_ es el LOF con el signo cambiado; se reinvierte.
score_lof = -lof.negative_outlier_factor_
print("Local Outlier Factor")
print("  Tamaño del vecindario : ", lof.n_neighbors)
print("  Observaciones marcadas : %d (%.1f %%)" % (anom_lof.sum(), 100 * anom_lof.mean()))
print("  Factor LOF: min = %.2f | mediana = %.2f | max = %.2f"
      % (score_lof.min(), np.median(score_lof), score_lof.max()))
# Sensibilidad al parámetro k: se comprueba cuánto cambia el conjunto detectado
# al variar el tamaño del vecindario.
print("\nSensibilidad al número de vecinos (solapamiento con k = 20):")
base_lof = anom_lof
for k in [5, 10, 20, 35, 50]:
    m = LocalOutlierFactor(n_neighbors=k, contamination=0.05).fit_predict(Z) == -1
    jac = (m & base_lof).sum() / (m | base_lof).sum()
    print("  k = %2d: %3d marcadas, índice de Jaccard con k=20: %.3f" % (k, m.sum(), jac))
```

```
Local Outlier Factor
  Tamaño del vecindario : 20
  Observaciones marcadas : 107 (5.0 %)
  Factor LOF: min = 0.95 | mediana = 1.07 | max = 4.50
Sensibilidad al número de vecinos (solapamiento con k = 20):
  k = 5: 107 marcadas, índice de Jaccard con k=20: 0.216
  k = 10: 107 marcadas, índice de Jaccard con k=20: 0.446
  k = 20: 107 marcadas, índice de Jaccard con k=20: 1.000
  k = 35: 107 marcadas, índice de Jaccard con k=20: 0.529
  k = 50: 107 marcadas, índice de Jaccard con k=20: 0.427
```

5.6 Comparación de los cinco detectores

Los cinco métodos ya han emitido su veredicto. La pregunta pendiente es cuál de ellos resulta realmente útil, y para responderla se recupera la etiqueta **NSP** que se había apartado al comienzo.

Listado 20. Celda 5.5. Evaluación comparativa contra la verdad de referencia externa

```
# Celda 5.5. Evaluación comparativa contra la verdad de referencia externa.
# Objetivo: medir la utilidad real de cada detector usando NSP, que ningún
# modelo ha visto. Se calcula el porcentaje de patológicos entre los marcados,
# que actúa como precisión del detector; el lift, que es ese porcentaje
# dividido por la tasa base; y el recall, que es la fracción del total de
# casos patológicos que el detector logra recuperar.
# Salidas: el DataFrame tabla_anomalías.
DETECTORES = {
    "Z-score (|z|>3)": anom_zscore,
    "Tukey (1.5 RIC)": anom_iqr,
    "Mahalanobis MCD (5 %)": anom_mahalanobis,
    "Isolation Forest (5 %)": anom_iforest,
    "LOF k=20 (5 %)": anom_lof,
}
tasa_base = float((y_nsp == 3).mean()) # prevalencia de patológicos: 8,3 %
total_patologicos = int((y_nsp == 3).sum())
filas = []
for nombre, mascara in DETECTORES.items():
    n = int(mascara.sum())
    prec = float((y_nsp[mascara] == 3).mean()) # precisión sobre NSP=3
    prec_23 = float((y_nsp[mascara] >= 2).mean()) # precisión sobre no normales
    rec = float((y_nsp[mascara] == 3).sum() / total_patologicos)
    filas.append({
        "Metodo": nombre,
        "Marcadas": n,
        "% del conjunto": round(100 * n / len(Z), 1),
        "% patológicos": round(100 * prec, 1),
        "Lift": round(prec / tasa_base, 2),
        "% no normales": round(100 * prec_23, 1),
        "Recall NSP=3": round(100 * rec, 1),
    })
tabla_anomalías = pd.DataFrame(filas).set_index("Metodo")
print("=" * 92)
print("EVALUACION DE LOS DETECTORES CONTRA LA ETIQUETA NSP")
print("=" * 92)
print("Tasa base de casos patológicos: %.1f %% (%d de %d)\n"
      % (100 * tasa_base, total_patologicos, len(y_nsp)))
print(tabla_anomalías.to_string())
print("\nUn lift de 1.00 significa que el detector no aporta nada sobre el azar.")
RES["tabla_anomalías"] = tabla_anomalías.reset_index().to_dict("records")
RES["tasa base nsp3"] = round(tasa_base, 4)
```

```
=====
EVALUACION DE LOS DETECTORES CONTRA LA ETIQUETA NSP
=====
Tasa base de casos patológicos: 8.3 % (176 de 2126)
Metodo      Marcadas % del conjunto % patológicos Lift % no normales Recall NSP=3
Z-score (|z|>3)      343          16.1          40.5 4.90          49.0          79.0
Tukey (1.5 RIC)     1220          57.4          13.7 1.65          30.4          94.9
Mahalanobis MCD (5 %) 106           5.0          41.5 5.01          49.1          25.0
Isolation Forest (5 %) 107           5.0          54.2 6.55          60.7          33.0
[Salida recortada: 3 líneas más. La salida completa está en el cuaderno del repositorio.]
```

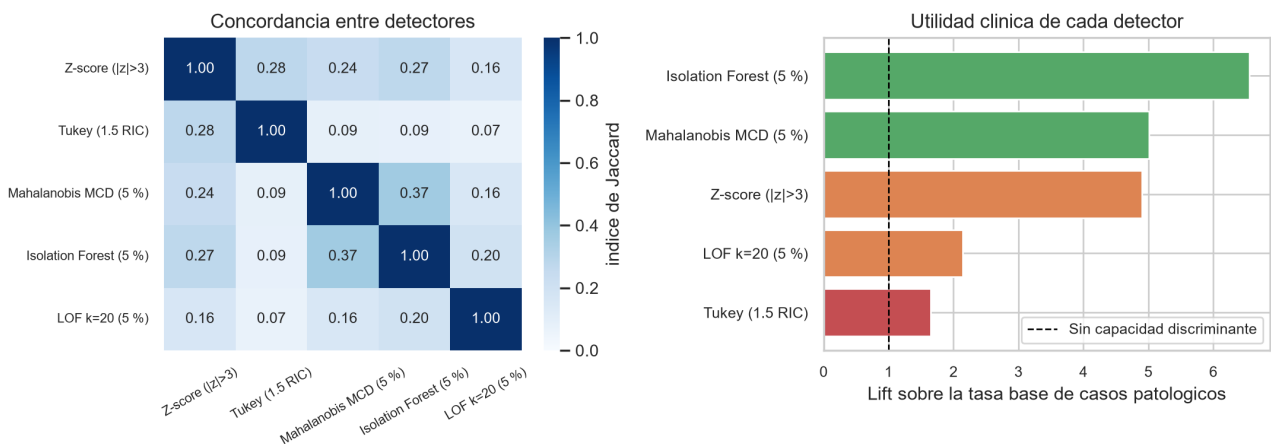
Listado 21. Celda 5.6. Concordancia entre detectores y visualizacion

```
# Celda 5.6. Concordancia entre detectores y visualizacion.
# Objetivo: medir cuanto coinciden los metodos entre si mediante el indice de
# Jaccard, definido como el tamano de la interseccion dividido por el de la
# union; visualizar las anomalias sobre la proyeccion PCA; y comparar el lift
# de forma grafica.
# Salidas: las figuras fig_anomalias_jaccard y fig_anomalias_pca.
nombres = list(DETECTORES.keys())
jaccard = np.ones((len(nombres), len(nombres)))
for i, ni in enumerate(nombres):
    for j, nj in enumerate(nombres):
        a, b = DETECTORES[ni], DETECTORES[nj]
        jaccard[i, j] = (a & b).sum() / max((a | b).sum(), 1)
fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4.0))
sns.heatmap(pd.DataFrame(jaccard, index=nombres, columns=nombres), annot=True,
            fmt=".2f", cmap="Blues", vmin=0, vmax=1, ax=a1,
            cbar_kws={"label": "indice de Jaccard"}, annot_kws={"size": 9})
a1.set_title("Concordancia entre detectores")
a1.tick_params(axis="x", rotation=30, labels=8)
a1.tick_params(axis="y", labels=8)
orden = tabla_anomalias["Lift"].sort_values()
colores = ["#C44E52" if v < 2 else "#DD8452" if v < 5 else "#5A868" for v in orden]
a2.barh(orden.index, orden.values, color=colores)
a2.axvline(1, color="black", ls="--", lw=1, label="Sin capacidad discriminante")
a2.set_xlabel("Lift sobre la tasa base de casos patologicos")
a2.set_title("Utilidad clinica de cada detector")
a2.tick_params(labels=9)
a2.legend(fontsize=9)
plt.tight_layout()
guardar("fig_anomalias_jaccard")
# Proyeccion PCA de las anomalias de los tres metodos multivariantes.
fig, ejes = plt.subplots(1, 3, figsize=(12, 3.9))
for eje, nombre in zip(ejes, ["Mahalanobis MCD (5 %)", "Isolation Forest (5 %)", "LOF k=20 (5 %)"]):
    m = DETECTORES[nombre]
    eje.scatter(Z2[~m, 0], Z2[~m, 1], s=6, c="#888888", label="Normal", alpha=0.6)
    eje.scatter(Z2[m, 0], Z2[m, 1], s=16, c="#C44E52", label="Anomalia", edgecolor="k", lw=0.2)
    eje.set_title("%s\n%0f %% de patologicos entre las marcadas"
                  % (nombre, tabla_anomalias.loc[nombre, "% patologicos"]), fontsize=10)
    eje.set_xlabel("CP1 (%0f %% var.)" % (100 * pca.explained_variance_ratio_[0]))
    eje.set_ylabel("CP2 (%0f %% var.)" % (100 * pca.explained_variance_ratio_[1]))
    eje.legend(fontsize=9, markerscale=1.6)
fig.suptitle("Anomalias detectadas sobre las dos primeras componentes principales", fontsize=12)
plt.tight_layout()
guardar("fig_anomalias_pca")
print("Indice de Jaccard entre los tres metodos multivariantes:")
print(" Isolation Forest y Mahalanobis: %.3f" % jaccard[nombres.index("Isolation Forest (5 %)",
nombres.index("Mahalanobis MCD (5 %)"))])
print(" Isolation Forest y LOF      : %.3f" % jaccard[nombres.index("Isolation Forest (5 %)",
nombres.index("LOF k=20 (5 %)"))])
print(" Mahalanobis y LOF          : %.3f" % jaccard[nombres.index("Mahalanobis MCD (5 %)",
nombres.index("LOF k=20 (5 %)"))])
RES["jaccard_if_lof"] = float(round(jaccard[nombres.index("Isolation Forest (5 %)",
nombres.index("LOF k=20 (5 %)")], 3))
```

Indice de Jaccard entre los tres metodos multivariantes:
 Isolation Forest y Mahalanobis: 0.365
 Isolation Forest y LOF : 0.196
 Mahalanobis y LOF : 0.158

Figura 7

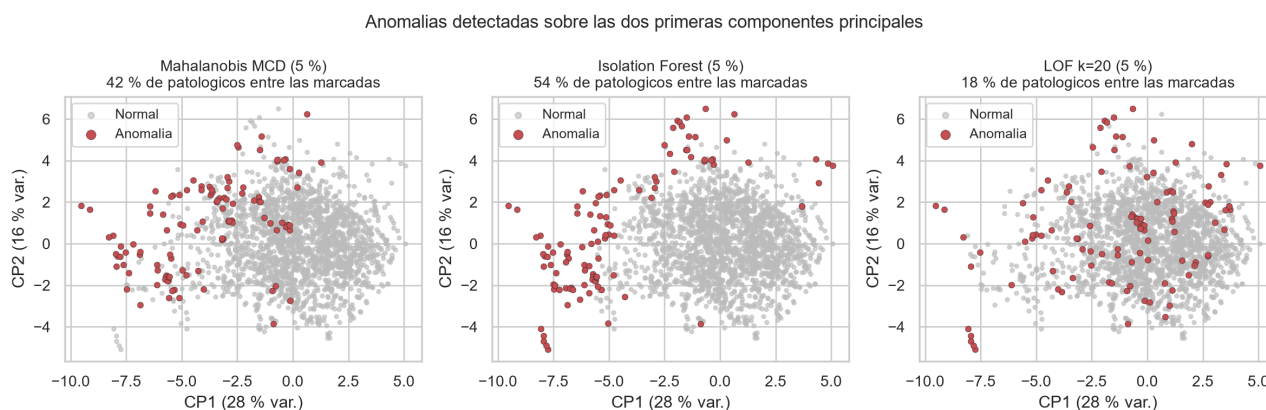
Concordancia entre detectores y utilidad clínica de cada uno



Nota. El índice de Jaccard mide la proporción de detecciones compartidas entre cada par de métodos. El lift es el cociente entre la proporción de casos patológicos entre los registros marcados y la tasa base del 8,3 %. Elaboración propia.

Figura 8

Anomalías detectadas por los tres métodos multivariantes



Nota. Proyección sobre las dos primeras componentes principales, que explican en conjunto el 43,3 % de la varianza. Elaboración propia.

La evaluación externa ordena los cinco detectores de forma inequívoca. Isolation Forest es el mejor con diferencia: de las 107 observaciones que marca, el 54,2 % son casos patológicos, frente a una tasa base del 8,3 %, lo que supone un lift de 6,55. Recupera además el 33 % de todos los casos patológicos del conjunto examinando solamente el 5 % de los registros, de modo que, en términos operativos, una revisión manual de ese 5 % encontraría uno de cada dos casos graves.

La distancia de Mahalanobis robusta queda en segundo lugar, con un lift de 5,01, resultado notable si se tiene en cuenta que su supuesto de normalidad está claramente violado; el estimador robusto compensa buena parte del problema. El Z-score alcanza un lift alto, de 4,90, pero a un coste inaceptable, ya que marca el 16 % del conjunto, y su recall del 79 % resulta engañoso porque se consigue señalando una de cada seis observaciones.

El factor local de atipicidad es el peor de los métodos multivariantes, con un lift de 2,14. El motivo es estructural: LOF busca anomalías locales y en un espacio de veinte dimensiones la noción de densidad local se diluye. A ello se suma que los trazados patológicos de este conjunto no forman puntos aislados sino una región periférica relativamente poblada, que LOF interpreta como un vecindario legítimo. La regla de Tukey, por último, no sirve en absoluto: su lift es de 1,65 marcando el 57 % del conjunto.

Hay un hallazgo transversal que conviene subrayar. Los índices de Jaccard son bajos en toda la matriz, y en particular Isolation Forest y LOF comparten apenas el 20 % de sus detecciones. Dicho de otro modo, ser una anomalía no es una propiedad absoluta del dato sino una propiedad relativa al método, y por eso la elección del detector no puede hacerse sin un criterio externo de utilidad como el que aquí aporta la etiqueta **NSP**.

Se adopta en consecuencia Isolation Forest al 5 % como detector de referencia. Sus 107 anomalías no se eliminan del conjunto, porque en un problema clínico los valores atípicos son la señal de interés y no ruido que convenga limpiar. Se conservan marcadas, y en la sección siguiente se comprueba que el agrupamiento las ubica efectivamente en un grupo propio.

6. Técnicas de agrupamiento

6.1 Planteamiento

El agrupamiento busca particionar las observaciones en grupos internamente homogéneos y mutuamente diferenciados, sin recurrir a ninguna etiqueta. Se aplican tres algoritmos de familias distintas.

K-Means es un método particional que minimiza la inercia dentro de cada grupo, tiende a producir grupos de forma esférica y tamaño parecido y exige fijar de antemano el número de grupos. DBSCAN pertenece a la familia de los métodos basados en densidad: busca regiones densas separadas por regiones vacías, admite grupos de forma arbitraria, no obliga a fijar su número y etiqueta como ruido aquello que no encaja en ninguno. El agrupamiento jerárquico aglomerativo con criterio de Ward construye una jerarquía de fusiones sucesivas que minimizan el incremento de varianza, produce grupos compactos y anidados y permite decidir el número de grupos a posteriori, al cortar el árbol a la altura deseada.

Para valorar los resultados se emplean dos tipos de métricas. Las de validación interna se calculan solo con los datos, y son el coeficiente de silueta, que se mueve entre menos uno y uno y mide la cohesión frente a la separación; el índice de Davies-Bouldin, que compara dispersión y separación y conviene que sea bajo; y el índice de Calinski-Harabasz, que relaciona la varianza entre grupos con la varianza interna y conviene que sea alto. Las de validación externa contrastan la partición con la etiqueta **NSP**, que no interviene en el ajuste, y son el índice de Rand ajustado y la información mutua normalizada; un índice de Rand ajustado igual a cero equivale a una partición completamente aleatoria.

6.2 K-Means: elección del número de grupos

Listado 22. Celda 6.1. Barrido del número de grupos k para K-Means

```
# Celda 6.1. Barrido del número de grupos k para K-Means.
# Objetivo: evaluar k entre 2 y 10 con cuatro criterios de validación interna y
# dos de validación externa, para decidir el número de grupos con evidencia y
# no por intuición.
# Nota: n_init = 20 relanza el algoritmo veinte veces con inicializaciones
# distintas y conserva la mejor, lo que mitiga su dependencia del arranque
# aleatorio, ya que K-Means solo garantiza alcanzar un óptimo local.
# Salidas: el DataFrame barrido_k.
RANGO_K = range(2, 11)
registros = []
for k in RANGO_K:
    modelo = KMeans(n_clusters=k, n_init=20, random_state=SEMILLA).fit(Z)
    etiquetas = modelo.labels_
    registros.append({
        "k": k,
        "Inercia": round(modelo.inertia_, 1),
        "Silueta": round(silhouette_score(Z, etiquetas), 4),
        "Davies-Bouldin": round(davies_bouldin_score(Z, etiquetas), 4),
        "Calinski-Harabasz": round(calinski_harabasz_score(Z, etiquetas), 1),
        "ARI vs NSP": round(adjusted_rand_score(y_nsp, etiquetas), 4),
        "NMI vs NSP": round(normalized_mutual_info_score(y_nsp, etiquetas), 4),
    })
barrido_k = pd.DataFrame(registros).set_index("k")
print("Barrido del número de grupos (validación interna y externa):\n")
print(barrido_k.to_string())
# Detección analítica del código de la curva de inercia. Se define como el punto
# de máxima distancia perpendicular a la recta que une el primer y el último
# valor de la curva, lo que evita la lectura subjetiva del gráfico.
ks = np.array(list(RANGO_K), dtype=float)
inercias = barrido_k["Inercia"].values.astype(float)
p1 = np.array([ks[0], inercias[0]])
p2 = np.array([ks[-1], inercias[-1]])
direccion = (p2 - p1) / np.linalg.norm(p2 - p1)
distancias = []
for x, yv in zip(ks, inercias):
    v = np.array([x, yv]) - p1
    distancias.append(np.linalg.norm(v - np.dot(v, direccion) * direccion))
k_codo = int(ks[int(np.argmax(distancias))])
print("\nCódigo detectado analíticamente en k =", k_codo)
RES["barrido_k"] = barrido_k.reset_index().to_dict("records")
RES["k_codo"] = k_codo
```

```

Barrido del numero de grupos (validacion interna y externa):
Inercia Silueta Davies-Bouldin Calinski-Harabasz ARI vs NSP NMI vs NSP
k
2 35166.1 0.1832 2.0107 444.2 0.0160 0.0864
3 31323.9 0.1572 1.9233 379.4 0.2142 0.2224
4 28241.2 0.1543 1.8186 357.6 0.0908 0.1822
5 26076.2 0.1610 1.5396 334.4 0.0883 0.1780
6 24366.9 0.1522 1.6241 315.9 0.1224 0.2257
7 22783.4 0.1568 1.4705 305.9 0.1261 0.2236
8 21572.2 0.1479 1.5920 293.8 0.1012 0.2172
[Salida recortada: 4 lineas mas. La salida completa esta en el cuaderno del repositorio.]

```

Listado 23. Celda 6.2. Representacion grafica de los criterios de seleccion de k

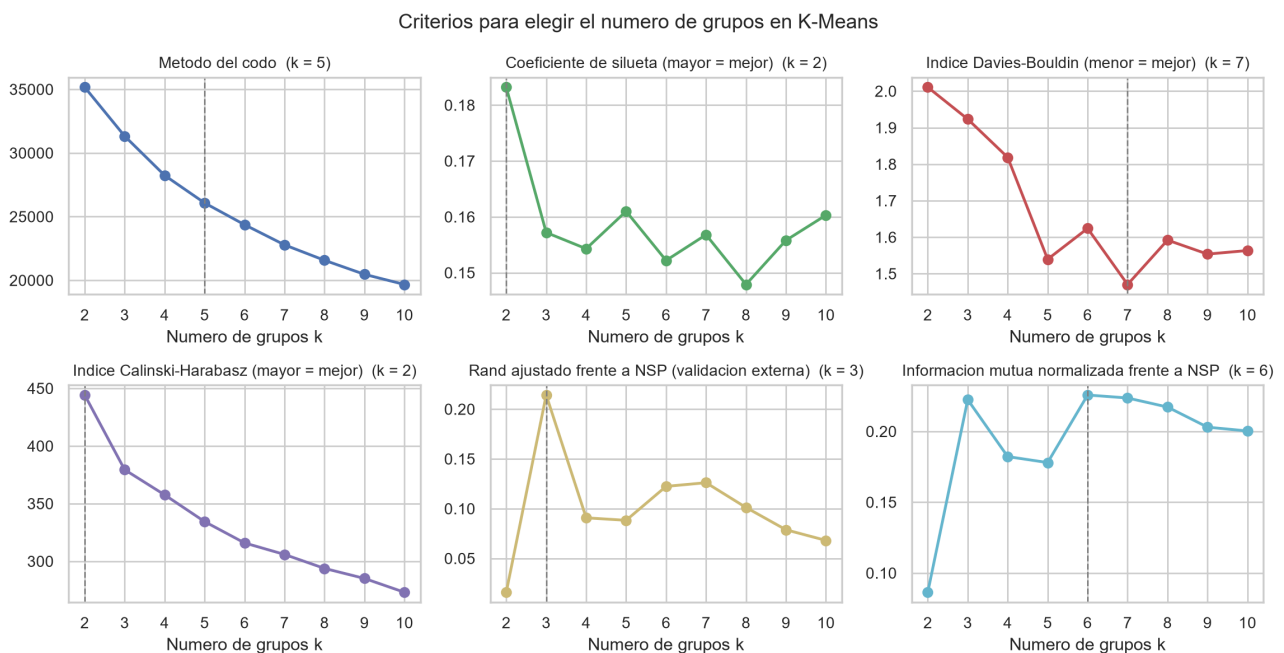
```

# Celda 6.2. Representacion grafica de los criterios de seleccion de k.
# Objetivo: mostrar en un solo panel los cuatro criterios internos y los dos
# externos, para que el conflicto entre ellos resulte visible de un vistazo.
# Salidas: la figura fig_kmeans_seleccion.
fig, ejes = plt.subplots(2, 3, figsize=(11.5, 6.0))
paneles = [
    ("Inercia", "Metodo del codo", "#4C72B0", None),
    ("Silueta", "Coeficiente de silueta (mayor = mejor)", "#55A868", "max"),
    ("Davies-Bouldin", "Indice Davies-Bouldin (menor = mejor)", "#C44E52", "min"),
    ("Calinski-Harabasz", "Indice Calinski-Harabasz (mayor = mejor)", "#8172B2", "max"),
    ("ARI vs NSP", "Rand ajustado frente a NSP (validacion externa)", "#CCB974", "max"),
    ("NMI vs NSP", "Informacion mutua normalizada frente a NSP", "#64B5CD", "max"),
]
for eje, (columna, titulo, color, optimo) in zip(ejes.flatten(), paneles):
    serie = barrido_k[columna]
    eje.plot(serie.index, serie.values, "o-", color=color, lw=1.8)
    # Se marca el k optimo segun ese criterio concreto.
    if optimo == "max":
        k_opt = serie.idxmax()
    elif optimo == "min":
        k_opt = serie.idxmin()
    else:
        k_opt = k_codo
    eje.axvline(k_opt, ls="--", color="grey", lw=1)
    eje.set_title(titulo + " (k = %d)" % k_opt, fontsize=10)
    eje.set_xlabel("Numero de grupos k")
    eje.set_xticks(list(RANGO_K))
fig.suptitle("Criterios para elegir el numero de grupos en K-Means", fontsize=13)
plt.tight_layout()
guardar("fig_kmeans_seleccion")

```

Figura 9

Criterios internos y externos para elegir el número de grupos



Nota. La línea vertical discontinua señala el valor óptimo de k según cada criterio por separado. Los dos paneles inferiores corresponden a la validación externa frente a la etiqueta NSP. Elaboración propia.

Los criterios no coinciden entre sí, y esa discrepancia constituye en sí misma el hallazgo más instructivo del apartado. La curva de inercia decrece de forma suave, sin codo pronunciado; el criterio analítico lo sitúa en k igual a 5, pero la curvatura es débil, lo que indica que no existe una estructura de grupos fuertemente separados. El coeficiente de silueta alcanza su máximo en k igual a 2, con un valor de 0,183, y todos los valores del barrido quedan por debajo de 0,19,

cuando en un conjunto con grupos nítidos se esperarían valores superiores a 0,5. El índice de Davies-Bouldin alcanza su mínimo en k igual a 7, con 1,47, y el de Calinski-Harabasz decrece de forma monótona, favoreciendo por tanto el menor k posible. Los tres criterios internos apuntan, en definitiva, a tres valores distintos: 5, 2 y 2.

La validación externa, en cambio, sí resulta concluyente. El índice de Rand ajustado alcanza su máximo inequívoco en k igual a 3, con 0,214, y cae a menos de la mitad en k igual a 4, donde vale 0,091. La información mutua normalizada es prácticamente máxima también ahí, con 0,222 frente a los 0,226 de k igual a 6, un valor que requiere el doble de grupos para no ganar nada.

Se adopta en consecuencia k igual a 3, por dos razones independientes que apuntan al mismo valor. La razón primaria es de dominio: el problema clínico está definido sobre tres estados fetales, normal, sospechoso y patológico, de manera que elegir tres grupos responde a la estructura conocida del fenómeno y no a los datos, lo que evita cualquier circularidad. La razón secundaria es la confirmación externa: que el índice de Rand ajustado alcance su máximo precisamente en tres grupos indica que esa estructura de tres niveles existe realmente en los descriptores numéricos y no es solo una convención médica.

Conviene señalar de forma explícita que el coeficiente de silueta habría llevado a elegir dos grupos, una partición cuyo índice de Rand ajustado resulta ser prácticamente nulo, de 0,016. Es un aviso importante: los índices internos miden geometría, no significado.

6.3 Perfilado e interpretación de los grupos

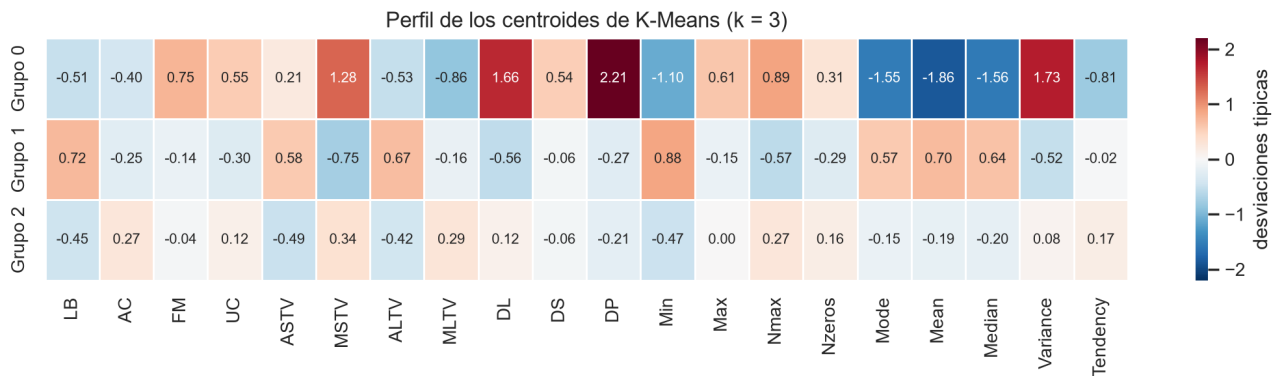
Listado 24. Celda 6.3. Ajuste final de K-Means y perfilado de los grupos

```
# Celda 6.3. Ajuste final de K-Means y perfilado de los grupos.
# Objetivo: ajustar el modelo definitivo y caracterizar cada grupo. Un
# agrupamiento sin interpretacion no pasa de ser un vector de numeros: el
# valor del analisis esta en explicar que distingue a cada grupo.
# Salidas: las etiquetas etq_kmeans, el DataFrame centroides y la figura
# fig_kmeans_perfil.
K_ELEGIDO = 3
kmeans = KMeans(n_clusters=K_ELEGIDO, n_init=50, random_state=SEMILLA).fit(Z)
etq_kmeans = kmeans.labels_
# Los centroides estan en unidades estandarizadas, de modo que cada valor indica
# cuantas desviaciones tipicas por encima o por debajo de la media general se
# situa el grupo en esa variable.
centroides = pd.DataFrame(kmeans.cluster_centers_, columns=VARIABLES_MODELO)
centroides.index = ["Grupo %d" % i for i in range(K_ELEGIDO)]
tamanos = pd.Series(etq_kmeans).value_counts().sort_index()
print("Tamano de los grupos:")
for i, n in tamanos.items():
    print(" Grupo %d: %d observaciones (%.1f %%)" % (i, n, 100 * n / len(Z)))
print("\nSilueta global: %.4f" % silhouette_score(Z, etq_kmeans))
plt.figure(figsize=(11, 3.1))
sns.heatmap(centroides, annot=True, fmt=".2f", cmap="RdBu_r", center=0,
            vmin=-2.2, vmax=2.2, linewidths=0.4, annot_kws={"size": 8},
            cbar_kws={"label": "desviaciones tipicas"})
plt.title("Perfil de los centroides de K-Means (k = 3)")
plt.tight_layout()
guardar("fig_kmeans_perfil")
print("\nVariables mas características de cada grupo:\n")
for g in range(K_ELEGIDO):
    fila = centroides.loc["Grupo %d" % g]
    destacadas = fila.reindex(fila.abs().sort_values(ascending=False).index).head(6)
    texto = ", ".join(["%s %.2f" % (v, x) for v, x in destacadas.items()])
    print(" Grupo %d (n = %d): %s" % (g, tamanos[g], texto))
RES["tamanos_kmeans"] = {int(i): int(n) for i, n in tamanos.items()}
RES["silueta_kmeans"] = round(float(silhouette_score(Z, etq_kmeans)), 4)
RES["centroides_kmeans"] = centroides.round(2).to_dict("index")
```

```
Tamano de los grupos:
Grupo 0: 204 observaciones (9.6 %)
Grupo 1: 835 observaciones (39.3 %)
Grupo 2: 1087 observaciones (51.1 %)
Silueta global: 0.1572
Variables mas características de cada grupo:
Grupo 0 (n = 204): DP +2.21, Mean -1.86, Variance +1.73, DL +1.66, Median -1.56, Mode -1.55
Grupo 1 (n = 835): Min +0.88, MSTV -0.75, LB +0.72, Mean +0.70, ALTV +0.67, Median +0.64
[Salida recortada: 1 lineas mas. La salida completa esta en el cuaderno del repositorio.]
```

Figura 10

Perfil de los centroides de K-Means con tres grupos



Nota. Cada celda indica cuántas desviaciones típicas por encima o por debajo de la media general se sitúa el grupo en esa variable. Elaboración propia.

Listado 25. Celda 6.4. Validacion externa: contraste de los grupos con el diagnostico NSP

```
# Celda 6.4. Validacion externa: contraste de los grupos con el diagnostico NSP.
# Objetivo: comprobar si los grupos hallados sin supervision se corresponden con
# los estados fetales que diagnosticaron los obstetras.
# Salidas: la tabla de contingencia y la figura fig_kmeans_nsp.
NOMBRES_NSP = {1: "Normal", 2: "Sospechoso", 3: "Patologico"}
contingencia = pd.crosstab(
    pd.Series(etq_kmeans, name="Grupo K-Means"),
    pd.Series([NOMBRES_NSP[v] for v in y_nsp], name="Diagnostico NSP"),
)
contingencia = contingencia[["Normal", "Sospechoso", "Patologico"]]
porcentajes = (100 * contingencia.div(contingencia.sum(axis=1), axis=0)).round(1)
print("Tabla de contingencia (conteos):\n")
print(contingencia.to_string())
print("\nComposicion de cada grupo (% por fila):\n")
print(porcentajes.to_string())
ari = adjusted_rand_score(y_nsp, etq_kmeans)
nmi = normalized_mutual_info_score(y_nsp, etq_kmeans)
# Pureza: proporcion de aciertos si a cada grupo se le asignara su clase mayoritaria.
pureza = contingencia.max(axis=1).sum() / contingencia.values.sum()
# Prueba chi-cuadrado de independencia entre grupo y diagnostico.
chi2, p_valor, gl, _ = stats.chi2_contingency(contingencia.values)
print("\nMetricas de validacion externa:")
print(" Indice de Rand ajustado (ARI) : %.4f" % ari)
print(" Informacion mutua normalizada : %.4f" % nmi)
print(" Pureza : %.4f" % pureza)
print(" Chi-cuadrado de independencia : %.1f (gl = %d), p = %.3e" % (chi2, gl, p_valor))
print("\nCon p < 0.001, la asociacion entre los grupos y el diagnostico clinico")
print("no es atribuible al azar.")
fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 3.9))
porcentajes.plot(kind="bar", stacked=True, ax=a1,
    color=["#55A868", "#DD8452", "#C44E52"], width=0.7)
a1.axhline(100 * (y_nsp == 1).mean(), ls="--", color="black", lw=1,
    label="Tasa base de normales (%.0f %%)" % (100 * (y_nsp == 1).mean()))
a1.set_ylabel("Composicion del grupo (%)")
a1.set_xlabel("")
a1.set_title("Composicion diagnostica de cada grupo")
a1.tick_params(axis="x", rotation=0)
a1.legend(fontsize=9, loc="lower right")
for g in range(K_ELEGIDO):
    m = etq_kmeans == g
    a2.scatter(Z2[m, 0], Z2[m, 1], s=7, alpha=0.6, label="Grupo %d (n=%d)" % (g, m.sum()))
centros_2d = pca.transform(kmeans.cluster_centers_)[0, :2]
a2.scatter(centros_2d[:, 0], centros_2d[:, 1], s=260, marker="X",
    c="black", edgecolor="white", lw=1.5, label="Centroides", zorder=5)
a2.set_xlabel("CP1 (%.0f %% var.)" % (100 * pca.explained_variance_ratio_[0]))
a2.set_ylabel("CP2 (%.0f %% var.)" % (100 * pca.explained_variance_ratio_[1]))
a2.set_title("Grupos sobre las componentes principales")
a2.legend(fontsize=9, markerscale=1.8)
plt.tight_layout()
guardar("fig_kmeans_nsp")
RES["ari_kmeans"] = round(float(ari), 4)
RES["nmi_kmeans"] = round(float(nmi), 4)
RES["pureza_kmeans"] = round(float(pureza), 4)
RES["contingencia_kmeans"] = contingencia.to_dict("index")
RES["porcentajes_kmeans"] = porcentajes.to_dict("index")
```

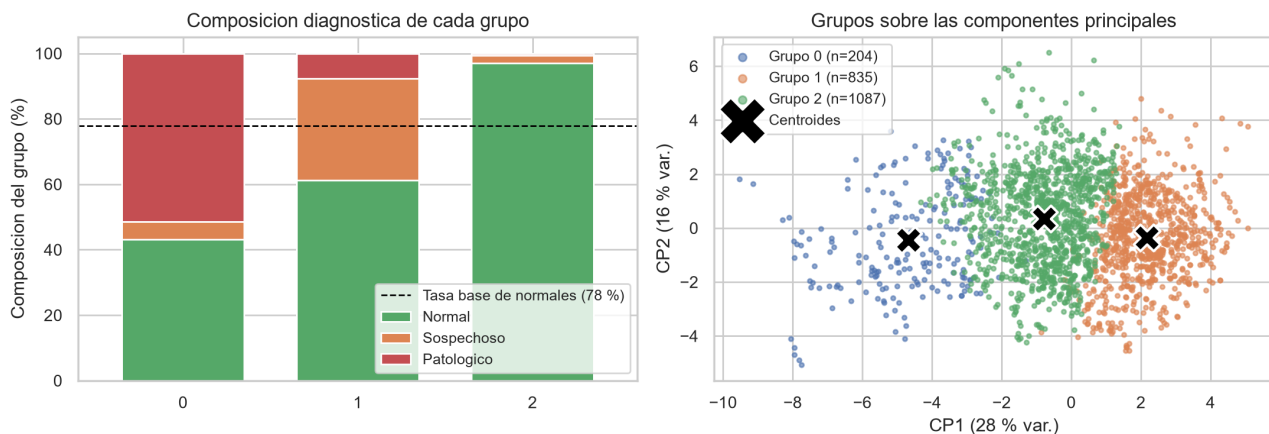
Tabla de contingencia (conteos):

Diagnostico NSP	Normal	Sospechoso	Patologico
Grupo K-Means			
0	88	11	105
1	512	259	64
2	1055	25	7

Composicion de cada grupo (% por fila):

Diagnostico NSP	Normal	Sospechoso	Patologico
0	88.0	11.0	105.0
1	512.0	259.0	64.0
2	1055.0	25.0	7.0

[Salida recortada: 13 lineas mas. La salida completa esta en el cuaderno del repositorio.]

Figura 11*Composición diagnóstica de los grupos y proyección sobre el plano principal*

Nota. A la izquierda, reparto de los tres diagnósticos dentro de cada grupo. A la derecha, los grupos sobre las dos primeras componentes principales, con los centroides marcados con una equis. Elaboración propia.

Listado 26. Celda 6.5. Relacion entre los grupos y las anomalías de la seccion 5

```
# Celda 6.5. Relacion entre los grupos y las anomalías de la seccion 5.
# Objetivo: comprobar si las dos técnicas, aplicadas de forma independiente,
# convergen. Si el agrupamiento aísla un grupo minoritario y ese mismo grupo
# concentra las anomalías de Isolation Forest, ambas están describiendo la
# misma estructura subyacente.
cruce = pd.crosstab(
    pd.Series(etq_kmeans, name="Grupo K-Means"),
    pd.Series(np.where(anom_iforest, "Anomalia IF", "Normal IF"), name="Isolation Forest"),
)
cruce_pct = (100 * cruce.div(cruce.sum(axis=1), axis=0)).round(1)
print("Anomalías de Isolation Forest repartidas por grupo de K-Means:\n")
print(cruce.to_string())
print("\nPorcentaje de anomalías dentro de cada grupo:\n")
print(cruce_pct["Anomalia IF"].to_string())
print("\nTasa global de anomalías: 5.0 %")
RES["anomalías por grupo"] = cruce_pct["Anomalia IF"].round(1).to_dict()
```

```
Anomalías de Isolation Forest repartidas por grupo de K-Means:
Isolation Forest  Anomalia IF  Normal IF
Grupo K-Means
0                  78          126
1                   8          827
2                  21         1066
Porcentaje de anomalías dentro de cada grupo:
Grupo K-Means
[Salida recortada: 5 líneas mas. La salida completa esta en el cuaderno del repositorio.]
```

El perfilado convierte tres etiquetas numéricas en tres fenotipos clínicos reconocibles.

El grupo 0 reúne 204 observaciones, el 9,6 % del conjunto, y corresponde al trazado comprometido. Es el grupo pequeño y extremo. Presenta **DP** en 2,21 y **DL** en 1,66 desviaciones típicas por encima de la media, lo que se traduce en abundantes deceleraciones prolongadas y ligeras; **Variance** en 1,73 y **MSTV** en 1,28, indicativos de una variabilidad errática; y valores muy bajos de tendencia central, con **Mean** en menos 1,86, **Median** en menos 1,56, **Mode** en menos 1,55 y **Min** en menos 1,10, es decir, bradicardia. Su composición diagnóstica es la más severa, con un 51,5 % de casos patológicos frente a la tasa base del 8,3 %. Este único grupo, que representa menos de una décima parte del conjunto, contiene el 60 % de todos los casos patológicos. Además, el 38 % de sus miembros fueron marcados como anomalía por Isolation Forest, cuando en los otros dos grupos esa proporción no llega al 2 %.

El grupo 1 reúne 835 observaciones, el 39,3 %, y corresponde a un patrón de variabilidad reducida con taquicardia relativa. Muestra **LB** en 0,72 y **Min** en 0,88, lo que indica una línea de base elevada; **ASTV** en 0,58 y **ALTV** en 0,67, que reflejan mucho tiempo con variabilidad anormal; y **MSTV** en menos 0,75, es decir, una variabilidad efectiva baja. Su composición es del 61,3 % de

casos normales pero también de un 31,0 % de sospechosos, casi el triple de la tasa base. Es, en definitiva, el grupo intermedio o de vigilancia.

El grupo 2 reúne 1 087 observaciones, el 51,1 %, y corresponde al trazado tranquilizador. Presenta **AC** en 0,27 y **MLTV** en 0,29, con aceleraciones presentes y buena variabilidad a largo plazo, junto a **ASTV** en menos 0,49 y **ALTV** en menos 0,42, esto es, poco tiempo anormal. Su composición es de un 97,1 % de casos normales, con apenas 7 patológicos.

El índice de Rand ajustado de 0,214 puede parecer modesto en abstracto, pero la prueba chi-cuadrado descarta el azar de forma contundente y la tabla de contingencia muestra un gradiente monótono de gravedad entre los grupos. El valor moderado tiene además una explicación clara: el agrupamiento separa muy bien los extremos, es decir, los grupos 0 y 2, pero no logra aislar la categoría de los sospechosos, que es intrínsecamente una zona de transición y no un fenotipo con frontera propia. Por otra parte, la convergencia entre las dos técnicas, dado que el grupo más anómalo según K-Means es también donde se concentran las anomalías de Isolation Forest, refuerza la conclusión de que la estructura hallada es real.

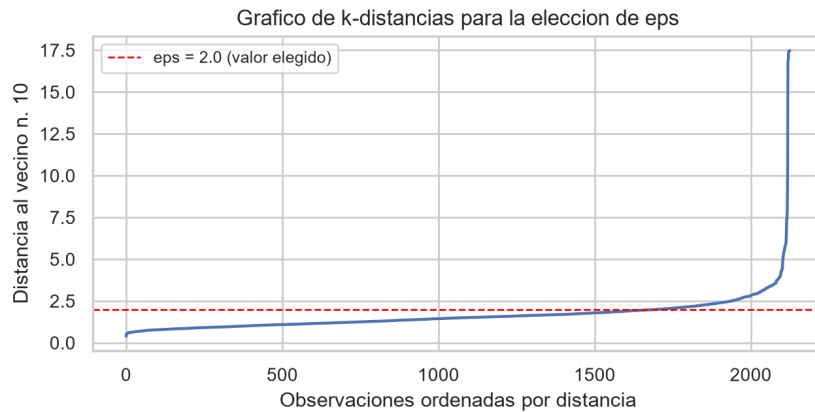
6.4 DBSCAN

Listado 27. Celda 6.6. DBSCAN: eleccion de eps mediante el grafico de k-distancias

```
# Celda 6.6. DBSCAN: eleccion de eps mediante el grafico de k-distancias.
# Objetivo: DBSCAN agrupa regiones densas y etiqueta como ruido, con el valor
# -1, lo que queda fuera. Necesita dos parametros: min_samples, que es el
# numero minimo de puntos para considerar densa una zona, y eps, que es el
# radio del vecindario. El valor de eps se elige con la heurística de Ester y
# colaboradores (1996): se ordena la distancia de cada punto a su k-esimo
# vecino y se busca el codo de la curva resultante.
# Nota: DBSCAN se aplica sobre el espacio reducido por PCA y no sobre las veinte
# variables originales, porque en dimension alta las distancias se concentran
# y la nocion de densidad pierde contraste.
# Salidas: la figura fig_dbscan_kdist y el valor EPS_ELEGIDO.
Z_dbscan = Z_pca80
MIN_SAMPLES = 10
print("Espacio de trabajo de DBSCAN:", Z_dbscan.shape[1], "componentes principales",
      "(%.0f %% de la varianza)" % (100 * np.cumsum(pca.explained_variance_ratio_)[Z_dbscan.shape[1] - 1]))
vecinos = NearestNeighbors(n_neighbors=MIN_SAMPLES).fit(Z_dbscan)
distancias, _ = vecinos.kneighbors(Z_dbscan)
k_dist = np.sort(distancias[:, -1])
plt.figure(figsize=(6.5, 3.4))
plt.plot(k_dist, lw=1.6, color="#4C72B0")
plt.axhline(2.0, color="red", ls="--", lw=1, label="eps = 2.0 (valor elegido)")
plt.xlabel("Observaciones ordenadas por distancia")
plt.ylabel("Distancia al vecino n. %d" % MIN_SAMPLES)
plt.title("Grafico de k-distancias para la eleccion de eps")
plt.legend(fontsize=9)
plt.tight_layout()
guardar("fig_dbscan_kdist")
print("\nPercentiles de la distancia al vecino n. %d:" % MIN_SAMPLES)
for p in [50, 75, 90, 95, 99]:
    print("    P%d = %.2f" % (p, np.percentile(k_dist, p)))
EPS_ELEGIDO = 2.0
```

```

Espacio de trabajo de DBSCAN: 9 componentes principales (83 % de la varianza)
Percentiles de la distancia al vecino n. 10:
P50 = 1.48
P75 = 1.88
P90 = 2.42
P95 = 2.94
P99 = 5.14
```

Figura 12*Gráfico de k-distancias para la elección del parámetro eps*

Nota. Distancia de cada observación a su décimo vecino más próximo, ordenada de menor a mayor. El codo de la curva orienta la elección del radio de vecindario. Elaboración propia.

Listado 28. Celda 6.7. Barrido de parámetros y ajuste final de DBSCAN

```
# Celda 6.7. Barrido de parámetros y ajuste final de DBSCAN.
# Objetivo: explorar la sensibilidad de DBSCAN al par (eps, min_samples) y
# ajustar el modelo definitivo. La silueta se calcula excluyendo el ruido,
# porque los puntos etiquetados con -1 no constituyen un grupo.
# Salidas: las etiquetas etq_dbscan y el DataFrame barrido_dbscan.
registros = []
for eps in [1.6, 1.8, 2.0, 2.2, 2.5, 3.0]:
    for ms in [10, 16, 20]:
        etiquetas = DBSCAN(eps=eps, min_samples=ms).fit_predict(Z_dbscan)
        n_grupos = len(set(etiquetas)) - (1 if -1 in etiquetas else 0)
        n_ruido = int((etiquetas == -1).sum())
        # La silueta requiere al menos dos grupos entre los puntos no ruidosos.
        if n_grupos > 1:
            sil = silhouette_score(Z_dbscan[etiquetas != -1], etiquetas[etiquetas != -1])
        else:
            sil = np.nan
        registros.append({
            "eps": eps, "min_samples": ms, "Grupos": n_grupos,
            "Ruido": n_ruido, "% ruido": round(100 * n_ruido / len(Z), 1),
            "Silueta (sin ruido)": round(sil, 4) if not np.isnan(sil) else None,
            "ARI vs NSP": round(adjusted_rand_score(y_nsp, etiquetas), 4),
        })
barrido_dbscan = pd.DataFrame(registros)
print("Sensibilidad de DBSCAN a sus dos parámetros:\n")
print(barrido_dbscan.to_string(index=False))
dbscan = DBSCAN(eps=EPS_ELEGIDO, min_samples=MIN_SAMPLES)
etq_dbscan = dbscan.fit_predict(Z_dbscan)
n_grupos_db = len(set(etq_dbscan)) - (1 if -1 in etq_dbscan else 0)
ruido_db = etq_dbscan == -1
print("\n" + "=" * 78)
print("DBSCAN definitivo (eps = %.1f, min_samples = %d)" % (EPS_ELEGIDO, MIN_SAMPLES))
print("=" * 78)
print(" Grupos encontrados :", n_grupos_db)
print(" Puntos de ruido : %d (%.1f %)" % (ruido_db.sum(), 100 * ruido_db.mean()))
print(" Tamaño de cada grupo:")
for g in sorted(set(etq_dbscan)):
    etiqueta = "ruido" if g == -1 else "grupo %d" % g
    print(" %-10s: %4d observaciones" % (etiqueta, (etq_dbscan == g).sum()))
RES["dbscan_n_grupos"] = int(n_grupos_db)
RES["dbscan_ruido"] = int(ruido_db.sum())
RES["barrido_dbscan"] = barrido_dbscan.to_dict("records")
```

Sensibilidad de DBSCAN a sus dos parámetros:

eps	min_samples	Grupos	Ruido	% ruido	Silueta (sin ruido)	ARI vs NSP
1.6	10	1	540	25.4	NaN	0.0704
1.6	16	1	716	33.7	NaN	0.0212
1.6	20	1	808	38.0	NaN	0.0150
1.8	10	2	300	14.1	0.3936	0.1597
1.8	16	1	439	20.6	NaN	0.1026
1.8	20	1	497	23.4	NaN	0.0785
2.0	10	2	213	10.0	0.3874	0.1965
2.0	16	2	268	12.6	0.3747	0.1784

[Salida recortada: 20 líneas mas. La salida completa esta en el cuaderno del repositorio.]

Listado 29. Celda 6.8. Interpretación de la salida de DBSCAN

```
# Celda 6.8. Interpretación de la salida de DBSCAN.
# Objetivo: contrastar los grupos y el ruido de DBSCAN con el diagnostico NSP y
# con las anomalías de Isolation Forest.
# Salidas: las tablas impresas y la figura fig_dbscan.
etiquetas_legibles = ["Ruido" if g == -1 else "Grupo %d" % g for g in etq_dbscan]
cont_db = pd.crosstab(
    pd.Series(etiquetas_legibles, name="DBSCAN"),
    pd.Series([NOMBRES_NSP[v] for v in y_nsp], name="Diagnostico NSP"),
)[["Normal", "Sospechoso", "Patologico"]]
print("Contingencia DBSCAN frente a NSP:\n")
print(cont_db.to_string())
print("\nComposición por fila (%):\n")
print((100 * cont_db.div(cont_db.sum(axis=1), axis=0)).round(1).to_string())
print("\nValidación externa: ARI = %.4f | NMI = %.4f"
```



```

% (adjusted_rand_score(y_nsp, etq_dbscan),
    normalized_mutual_info_score(y_nsp, etq_dbscan)))
jaccard_ruido = (ruido_db & anom_iforest).sum() / (ruido_db | anom_iforest).sum()
print("\nCoincidencia entre el ruido de DBSCAN y las anomalías de Isolation Forest:")
print(" Índice de Jaccard: %.3f" % jaccard_ruido)
print(" Puntos marcados por ambos metodos: %d" % (ruido_db & anom_iforest).sum())
fig, (a1, a2) = plt.subplots(1, 2, figsize=(11, 4.0))
for g in sorted(set(etq_dbscan)):
    m = etq_dbscan == g
    if g == -1:
        a1.scatter(Z2[m, 0], Z2[m, 1], s=14, c="#C44E52", marker="x",
            label="Ruido (n=%d)" % m.sum())
    else:
        a1.scatter(Z2[m, 0], Z2[m, 1], s=7, alpha=0.6, label="Grupo %d (n=%d)" % (g, m.sum()))
a1.set_xlabel("CP1"); a1.set_ylabel("CP2")
a1.set_title("Resultado de DBSCAN (eps = %.1f, min_samples = %d)" % (EPS_ELEGIDO, MIN_SAMPLES))
a1.legend(fontsize=9, markerscale=1.6)
colores_nsp = {1: "#55A868", 2: "#DD8452", 3: "#C44E52"}
for v in [1, 2, 3]:
    m = y_nsp == v
    a2.scatter(Z2[m, 0], Z2[m, 1], s=7, alpha=0.65, c=colores_nsp[v],
        label="%s (n=%d)" % (NOMBRES_NSP[v], m.sum()))
a2.set_xlabel("CP1"); a2.set_ylabel("CP2")
a2.set_title("Referencia: diagnostico real NSP")
a2.legend(fontsize=9, markerscale=1.8)
plt.tight_layout()
guardar("fig_dbscan")
RES["dbscan_ari"] = round(float(adjusted_rand_score(y_nsp, etq_dbscan)), 4)
RES["jaccard ruido if"] = round(float(jaccard_ruido), 3)

```

Contingencia DBSCAN frente a NSP:

Diagnostico NSP	Normal	Sospechoso	Patologico
DBSCAN			
Grupo 0	1537	272	76
Grupo 1	0	0	28
Ruido	118	23	72

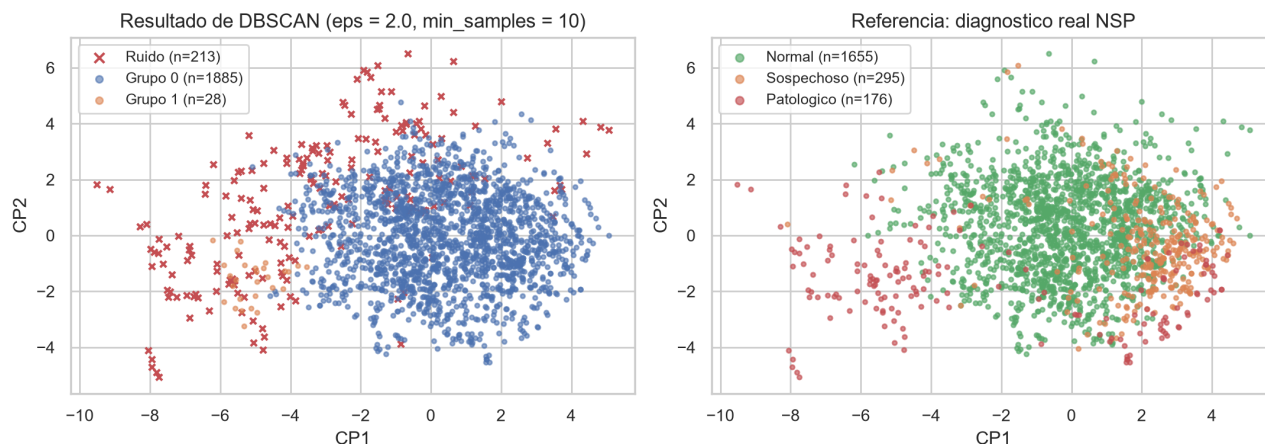
Composicion por fila (%):

Diagnostico NSP	Normal	Sospechoso	Patologico
-----------------	--------	------------	------------

[Salida recortada: 10 líneas mas. La salida completa esta en el cuaderno del repositorio.]

Figura 13

Resultado de DBSCAN frente al diagnóstico real



Nota. A la izquierda, los grupos y el ruido que identifica el algoritmo. A la derecha, las mismas observaciones coloreadas según la etiqueta NSP, que el algoritmo no ha utilizado. Elaboración propia.

DBSCAN no produce una partición útil del conjunto, y el motivo resulta informativo. Con los parámetros elegidos encuentra dos grupos y 213 puntos de ruido, pero el reparto es radicalmente asimétrico: un grupo absorbe 1 885 observaciones, el 88,7 % del conjunto, y el otro apenas 28.

El barrido de parámetros demuestra que este comportamiento es estructural y no consecuencia de una mala elección de eps. En once de las dieciocho combinaciones probadas DBSCAN devuelve un único grupo más ruido, y en las siete restantes el segundo grupo nunca supera unas pocas decenas de puntos. La razón es que los datos no tienen zonas de baja densidad que separen regiones densas, sino que forman una nube única cuya densidad decae de manera continua hacia la periferia. DBSCAN necesita valles y aquí no los hay.

Ahora bien, el grupo 1, formado por 28 observaciones, está compuesto íntegramente por casos patológicos. El algoritmo ha aislado un fenotipo extremo con una precisión perfecta, si bien cubre solo el 16 % de los patológicos del conjunto. El ruido también resulta informativo: de los 213 puntos etiquetados con menos uno, el 33,8 % son patológicos, cuatro veces la tasa base, y su índice de Jaccard con las anomalías de Isolation Forest es de 0,455, ya que comparten 100 puntos; se trata de la concordancia más alta observada entre dos métodos en todo el trabajo.

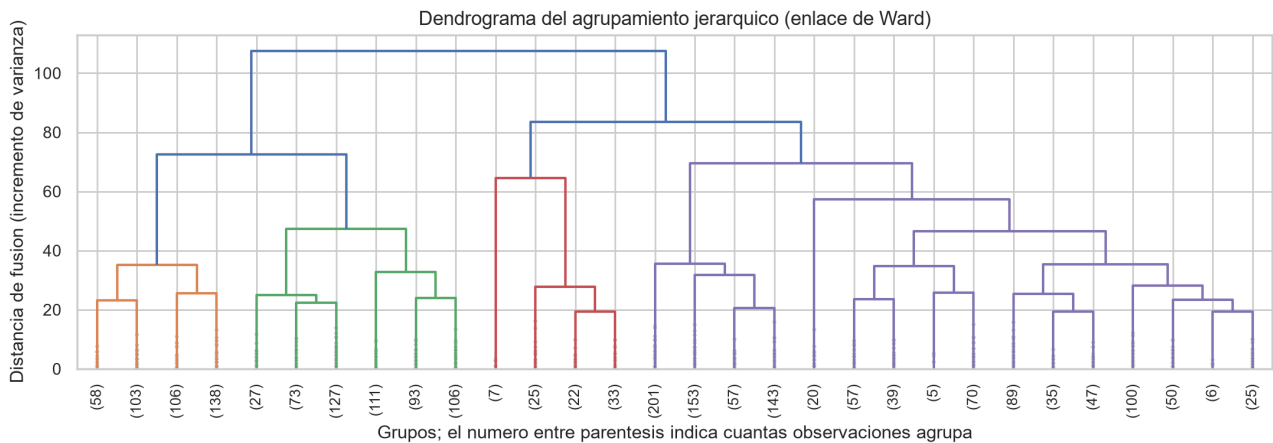
La conclusión es que en este conjunto DBSCAN rinde mejor como detector de anomalías que como algoritmo de agrupamiento. Su índice de Rand ajustado global, de 0,197, es comparable al de K-Means, pero se consigue por una vía distinta: no por particionar bien, sino por apartar correctamente los casos extremos. Resulta coherente, en ese sentido, que su información mutua normalizada, de 0,134, sea muy inferior a la de K-Means, que llega a 0,222, porque reparte mucha menos información sobre el diagnóstico al dejar el 88,7 % de los registros en un único grupo indiferenciado.

6.5 Agrupamiento jerárquico aglomerativo

Listado 30. Celda 6.9. Agrupamiento jerárquico: dendrograma y criterios de enlace

```
# Celda 6.9. Agrupamiento jerárquico: dendrograma y criterios de enlace.
# Objetivo: construir la jerarquía completa de fusiones y comparar tres
# criterios de enlace. El dendrograma permite decidir el número de grupos a
# posteriori, observando a que altura se producen las fusiones mas costosas.
# Salidas: la figura fig_dendrograma, las etiquetas etq_jerarquico y el
# DataFrame comparacion_enlaces.
# linkage() de SciPy calcula la jerarquía completa. El metodo de Ward minimiza
# el incremento de varianza intragrupo en cada fusion, que es el criterio mas
# proximo al que emplea K-Means.
matriz_enlace = linkage(Z, method="ward")
plt.figure(figsize=(11, 4.0))
dendrogram(matriz_enlace, truncate_mode="lastp", p=30, leaf_rotation=90,
            leaf_font_size=9, show_contracted=True, color_threshold=70)
plt.title("Dendrograma del agrupamiento jerárquico (enlace de Ward)")
plt.xlabel("Grupos; el número entre parentesis indica cuantas observaciones agrupa")
plt.ylabel("Distancia de fusion (incremento de varianza)")
plt.tight_layout()
guardar("fig_dendrograma")
# Comparacion de criterios de enlace. Se comprueba una advertencia clasica: los
# enlaces average y complete pueden producir particiones degeneradas con
# siluetas enganosamente altas.
filas = []
for enlace in ["ward", "average", "complete"]:
    etiquetas = AgglomerativeClustering(n_clusters=3, linkage=enlace).fit_predict(Z)
    tam = np.bincount(etiquetas)
    filas.append({
        "Enlace": enlace,
        "Tamanos": " / ".join(str(t) for t in sorted(tam, reverse=True)),
        "Grupo mayor (%)": round(100 * tam.max() / len(Z), 1),
        "Silueta": round(silhouette_score(Z, etiquetas), 4),
        "ARI vs NSP": round(adjusted_rand_score(y_nsp, etiquetas), 4),
    })
comparacion_enlaces = pd.DataFrame(filas).set_index("Enlace")
print("Comparacion de criterios de enlace con k = 3:\n")
print(comparacion_enlaces.to_string())
print("\nLos enlaces average y complete logran siluetas muy superiores a ward,")
print("pero dejando mas del 99 % de las observaciones en un unico grupo. Una")
print("silueta alta con una particion degenerada NO indica un buen agrupamiento.")
jerarquico = AgglomerativeClustering(n_clusters=3, linkage="ward")
etq_jerarquico = jerarquico.fit_predict(Z)
print("\nAgrupamiento jerárquico definitivo (Ward, k = 3):")
print("Tamanos      :", np.bincount(etq_jerarquico).tolist())
print("Silueta       : %.4f" % silhouette_score(Z, etq_jerarquico))
print("ARI vs NSP    : %.4f" % adjusted_rand_score(y_nsp, etq_jerarquico))
print("ARI vs K-Means: %.4f (concordancia entre ambas particiones)"
      % adjusted_rand_score(etq_kmeans, etq_jerarquico))
RES["comparacion_enlaces"] = comparacion_enlaces.reset_index().to_dict("records")
RES["ari_jerarquico"] = round(float(adjusted_rand_score(y_nsp, etq_jerarquico)), 4)
RES["ari_kmeans_vs_jerarquico"] = round(float(adjusted_rand_score(etq_kmeans, etq_jerarquico)), 4)
```

```
Comparacion de criterios de enlace con k = 3:
      Tamanos  Grupo mayor (%)  Silueta  ARI vs NSP
Enlace
ward      1097 / 942 / 87      51.6    0.1348    0.1793
average    2114 / 7 / 5      99.4    0.5188    0.0187
complete   2116 / 7 / 3      99.5    0.5749    0.0202
Los enlaces average y complete logran siluetas muy superiores a ward,
pero dejando mas del 99 % de las observaciones en un unico grupo. Una
silueta alta con una particion degenerada NO indica un buen agrupamiento.
[Salida recortada: 6 líneas mas. La salida completa esta en el cuaderno del repositorio.]
```

Figura 14*Dendrograma del agrupamiento jerárquico con enlace de Ward*

Nota. Se representan las últimas treinta fusiones. La altura de cada unión corresponde al incremento de varianza que provoca. Elaboración propia.

El dendrograma muestra un salto de altura claro en las últimas fusiones, compatible con dos o tres grupos, lo que respalda la elección de k igual a 3.

La comparación de criterios de enlace produce el aviso metodológico más importante de la sección. Los enlaces **average** y **complete** obtienen siluetas de 0,52 y 0,57 respectivamente, es decir, hasta cuatro veces mejores que las de Ward, que se queda en 0,13, pero lo consiguen dejando más del 99 % de las observaciones en un único grupo y aislando dos grupitos de entre 3 y 7 puntos. Sus índices de Rand ajustado son prácticamente nulos, de 0,02. Es la ilustración perfecta de que una métrica interna alta puede corresponder a un modelo inservible, porque la silueta premia particiones en las que casi todo está junto y unos pocos puntos muy alejados forman grupos triviales.

Con el enlace de Ward la partición es equilibrada, con 1 097, 942 y 87 observaciones, y su índice de Rand ajustado frente a **NSP** es de 0,179, algo inferior al de K-Means, que llega a 0,214. Curiosamente su información mutua normalizada es ligeramente superior, de 0,230 frente a 0,222, lo que indica que Ward capta algo más de información sobre el diagnóstico pero la reparte peor entre los grupos, que es justo lo que penaliza el índice de Rand. La concordancia entre ambas particiones es de 0,371, de modo que coinciden en lo esencial pero no son intercambiables, sobre todo en el trazado de la frontera entre los dos grupos grandes.

6.6 Comparación global de los tres algoritmos

Listado 31. Celda 6.10. Comparacion final de los tres algoritmos de agrupamiento

```
# Celda 6.10. Comparacion final de los tres algoritmos de agrupamiento.
# Objetivo: reunir en una sola tabla y en una sola figura los resultados de
# K-Means, DBSCAN y jerárquico, con metricas internas y externas.
# Salidas: el DataFrame tabla_clustering y la figura fig_clustering_comparacion.
def evaluar_agrupamiento(nombre, etiquetas, espacio, requiere_k, maneja_ruido):
    """Calcula el bloque de metricas de un agrupamiento.
    El ruido, con etiqueta -1, se excluye de las metricas internas, ya que por
    definicion no constituye un grupo; si se incluyera penalizaria de forma
    artificial a DBSCAN frente a los algoritmos que particionan todo.
    """
    validos = etiquetas != -1
    n_grupos = len(set(etiquetas[validos]))
    sil = silhouette_score(espacio[validos], etiquetas[validos]) if n_grupos > 1 else np.nan
    db = davies_bouldin_score(espacio[validos], etiquetas[validos]) if n_grupos > 1 else np.nan
    return {
        "Algoritmo": nombre,
        "Grupos": n_grupos,
        "Sin asignar": int((~validos).sum()),
        "Silueta": round(sil, 4) if not np.isnan(sil) else None,
```

```

"Davies-Bouldin": round(db, 4) if not np.isnan(db) else None,
"ARI vs NSP": round(adjusted_rand_score(y_nsp, etiquetas), 4),
"NMI vs NSP": round(normalized_mutual_info_score(y_nsp, etiquetas), 4),
"Requiere k": "Si" if requiere_k else "No",
"Detecta ruido": "Si" if maneja_ruido else "No",
}
tabla_clustering = pd.DataFrame([
    evaluar_agrupamiento("K-Means (k=3)", etq_kmeans, Z, True, False),
    evaluar_agrupamiento("DBSCAN (eps=2.0)", etq_dbscan, Z_dbscan, False, True),
    evaluar_agrupamiento("Jerarquico Ward (k=3)", etq_jerarquico, Z, True, False),
]).set_index("Algoritmo")
print(" " * 100)
print("COMPARACION FINAL DE LOS ALGORITMOS DE AGRUPAMIENTO")
print(" " * 100)
print(tabla_clustering.to_string())
fig, ejes = plt.subplots(1, 3, figsize=(12, 3.9))
particiones = [
    ("K-Means (k=3)", etq_kmeans),
    ("DBSCAN (eps=2.0)", etq_dbscan),
    ("Jerarquico Ward (k=3)", etq_jerarquico),
]
for eje, (nombre, etiquetas) in zip(ejes, particiones):
    for g in sorted(set(etiquetas)):
        m = etiquetas == g
        if g == -1:
            eje.scatter(Z2[m, 0], Z2[m, 1], s=12, c="#999999", marker="x", label="Ruido")
        else:
            eje.scatter(Z2[m, 0], Z2[m, 1], s=6, alpha=0.6, label="G%d" % g)
    eje.set_title("%s\nARI = %.3f" % (nombre, tabla_clustering.loc[nombre, "ARI vs NSP"]), fontsize=10)
    eje.set_xlabel("CP1"); eje.set_ylabel("CP2")
    eje.legend(fontsize=9, markerscale=1.8)
fig.suptitle("Los tres agrupamientos sobre el mismo plano de componentes principales", fontsize=12)
plt.tight_layout()
guardar("fig_clustering_comparacion")
RES["tabla_clustering"] = tabla_clustering.reset_index().to_dict("records")

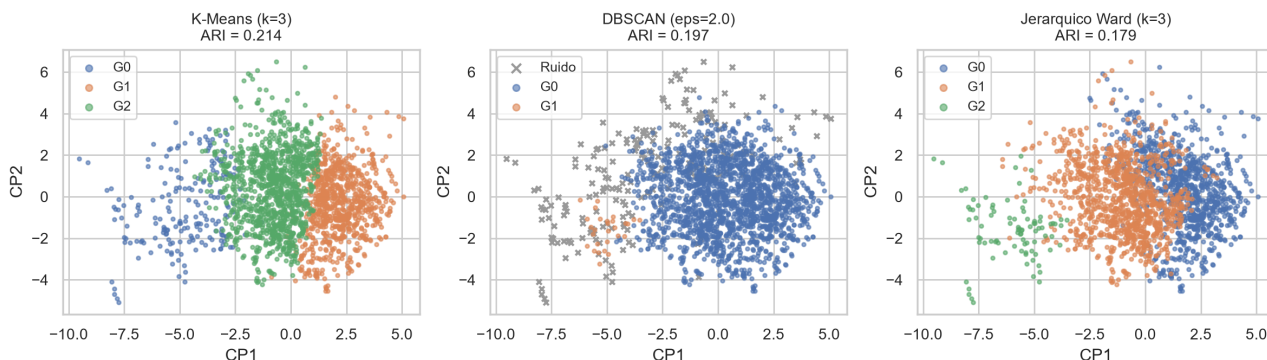
```

COMPARACION FINAL DE LOS ALGORITMOS DE AGRUPAMIENTO								
Algoritmo	Grupos	Sin asignar	Silueta	Davies-Bouldin	ARI vs NSP	NMI vs NSP	Requiere k	Detecta ruido
K-Means (k=3)	3	0	0.1572	1.9233	0.2142	0.2224	Si	No
DBSCAN (eps=2.0)	2	213	0.3874	0.7396	0.1965	0.1340	No	Si
Jerarquico Ward (k=3)	3	0	0.1348	2.0230	0.1793	0.2297	Si	No

Figura 15

Los tres agrupamientos sobre el mismo plano de componentes principales

Los tres agrupamientos sobre el mismo plano de componentes principales



Nota. Bajo el nombre de cada algoritmo se indica su índice de Rand ajustado frente a la etiqueta NSP. Elaboración propia.

7. Ventajas y desventajas de cada modelo

7.1 Modelos de detección de anomalías

El Z-score tiene a su favor que es trivial de calcular e interpretar y que su umbral posee un significado probabilístico directo. En su contra pesan tres limitaciones: asume normalidad, es univariante y por tanto ignora por completo las correlaciones entre variables, y su error acumulado crece con el número de columnas examinadas. En este conjunto marcó el 16,1 % de las observaciones y, aunque alcanzó un lift de 4,90, lo hizo a costa de demasiados falsos positivos.

La regla de Tukey no asume ninguna distribución, es robusta frente a valores extremos y constituye la base del diagrama de caja, que es una herramienta exploratoria muy valiosa. Sin

embargo, también es univariante y resulta inservible cuando las variables presentan fuerte asimetría o exceso de ceros, que es exactamente lo que ocurre aquí. Marcó el 57,4 % del conjunto con un lift de 1,65, de manera que no discrimina.

La distancia de Mahalanobis con estimación robusta es multivariante y tiene en cuenta la covarianza, y el estimador de determinante mínimo evita el efecto de enmascaramiento que padecen la media y la covarianza muestrales. Como contrapartida exige que la matriz de covarianzas sea invertible, requisito que obligó a eliminar la variable `width`, asume normalidad multivariante y su coste computacional crece de forma cúbica con el número de variables. Obtuvo un lift de 5,01 marcando el 5 % del conjunto, si bien el gráfico cuantil-cuantil dejó claro que su supuesto no se cumple.

Isolation Forest no asume ninguna distribución, su coste es lineal en el número de observaciones, escala bien en dimensión alta y tiene pocos hiperparámetros que ajustar. Sus inconvenientes son que la proporción de contaminación debe fijarse a priori, que el resultado varía con la semilla aleatoria y que sus cortes son siempre paralelos a los ejes, lo que puede dificultar la detección de estructuras oblicuas. Fue el mejor método, con un lift de 6,55, un 54,2 % de casos patológicos entre los marcados y un recall del 33 %.

El factor local de atipicidad es el único capaz de detectar anomalías locales y no impone ninguna forma global a los datos. En contra tiene una sensibilidad muy alta al número de vecinos, la degradación de la noción de densidad en dimensión alta y la imposibilidad de generalizar a datos nuevos salvo que se active explícitamente el modo de novedad. Resultó el peor de los métodos multivariantes, con un lift de 2,14.

7.2 Modelos de agrupamiento

K-Means es rápido y escalable, produce grupos fácilmente interpretables a través de sus centroides y converge siempre. A cambio exige fijar el número de grupos, supone que estos son esféricos y de tamaño similar, sus centroides no son robustos frente a valores atípicos y solo garantiza alcanzar óptimos locales. Fue el mejor de los tres, con un índice de Rand ajustado de 0,214 y tres fenotipos clínicamente coherentes.

DBSCAN no exige fijar el número de grupos, admite grupos de forma arbitraria e identifica el ruido de manera explícita. Sus desventajas son la enorme sensibilidad al par de parámetros `eps` y `min_samples`, el fracaso cuando la densidad varía de forma continua y la degradación en dimensión alta. Aquí produjo dos grupos muy desiguales, de 1 885 y 28 observaciones, más 213 puntos de ruido, y un índice de Rand ajustado de 0,197; resultó útil como detector pero no como partición.

El agrupamiento jerárquico produce la jerarquía completa, de modo que el número de grupos puede decidirse después, su dendrograma es muy interpretable y el resultado es determinista. En su contra están el coste cuadrático en memoria y tiempo, la imposibilidad de deshacer una fusión ya realizada y una fuerte dependencia del criterio de enlace. Con Ward obtuvo un índice de Rand ajustado de 0,179, mientras que los enlaces `average` y `complete` degeneraron pese a

exhibir siluetas mucho más altas.

7.3 Síntesis metodológica

De la comparación se desprenden tres lecciones transversales. La primera es que las métricas internas no bastan por sí solas. El coeficiente de silueta escogía dos grupos en K-Means, partición con un índice de Rand ajustado de 0,016, y premiaba los enlaces degenerados del jerárquico con valores de 0,52 y 0,57 pese a que dejaban el 99 % de los datos en un solo grupo. Sin una referencia externa o sin conocimiento del dominio, esos criterios habrían llevado a conclusiones erróneas.

La segunda es que los supuestos distribucionales importan más que la sofisticación del método. Los dos algoritmos que no asumen ninguna distribución, Isolation Forest y K-Means, fueron los mejores de su categoría, y lo fueron precisamente en un conjunto de datos fuertemente asimétrico.

La tercera es que la convergencia entre técnicas independientes constituye la mejor evidencia disponible. El grupo 0 de K-Means, el grupo 1 y el ruido de DBSCAN, y las anomalías de Isolation Forest apuntan al mismo subconjunto de trazados. Tres algoritmos con criterios matemáticos distintos coinciden porque existe una estructura real que describir.

8. Conclusiones

Listado 32. Celda 8.1. Resumen ejecutivo de resultados y volcado a disco

```
# Celda 8.1. Resumen ejecutivo de resultados y volcado a disco.
# Objetivo: recapitular las cifras clave y guardar todos los resultados en
# resultados.json, que es la fuente unica desde la que se genera el informe en
# PDF. De este modo ninguna cifra del informe se transcribe a mano y no puede
# desincronizarse del codigo.
# Salidas: el archivo resultados.json.
print("=" * 78)
print("RESUMEN EJECUTIVO")
print("=" * 78)
print("Conjunto de datos")
print("  Registros analizados      : %d (de %d en el archivo original)"
      % (RES["n_filas_final"], RES["n_filas_final"] + RES["n_filas_eliminadas"]))
print("  Variables descriptivas    : %d" % RES["n_variables_modelo"])
print("  Filas eliminadas          : %d artefactos de exportacion" % RES["n_filas_eliminadas"])
print("\nDeteccion de anomalias (mejor modelo: Isolation Forest al 5 %)")
mejor = max(RES["tabla_anomalias"], key=lambda r: r["Lift"])
print("  Observaciones marcadas    : %d" % mejor["Marcadas"])
print("  Patologicos entre ellas    : %.1f %% (tasa base %.1f %%)"
      % (mejor["% patologicos"], 100 * RES["tasa_base_nsp3"]))
print("  Lift                      : %.2f" % mejor["Lift"])
print("  Recall de casos NSP = 3    : %.1f %%" % mejor["Recall NSP=3"])
print("\nAgrupamiento (mejor modelo: K-Means con k = 3)")
print("  Silueta                   : %.4f" % RES["silueta_kmeans"])
print("  ARI frente a NSP          : %.4f" % RES["ari_kmeans"])
print("  NMI frente a NSP          : %.4f" % RES["nmi_kmeans"])
print("  Pureza                    : %.4f" % RES["pureza_kmeans"])
print("  Grupos                    : ", RES["tamanos_kmeans"])
def convertir(objeto):
    """Convierte tipos de NumPy a tipos nativos serializables por json."""
    if isinstance(objeto, (np.integer,)):
        return int(objeto)
    if isinstance(objeto, (np.floating,)):
        return float(objeto)
    if isinstance(objeto, np.ndarray):
        return objeto.tolist()
    return str(objeto)
with open("resultados.json", "w", encoding="utf-8") as fh:
    json.dump(RES, fh, ensure_ascii=False, indent=2, default=convertir)
print("\nResultados guardados en 'resultados.json' (%d claves)." % len(RES))
print("Figuras guardadas en '%s/' (%d archivos)."
      % (DIR_FIGURAS, len(list(DIR_FIGURAS.glob("*.png")))))
```

```
=====
RESUMEN EJECUTIVO
=====
Conjunto de datos
  Registros analizados      : 2126 (de 2129 en el archivo original)
  Variables descriptivas    : 20
  Filas eliminadas          : 3 artefactos de exportacion
Deteccion de anomalias (mejor modelo: Isolation Forest al 5 %)
  Observaciones marcadas    : 107
  Patologicos entre ellas    : 54.2 % (tasa base 8.3 %)
[Salida recortada: 12 líneas mas. La salida completa esta en el cuaderno del repositorio.]
```


8.1 Sobre el análisis exploratorio

El análisis exploratorio no fue un trámite previo, sino que determinó todas las decisiones posteriores. Permitted descubrir que las tres últimas filas del archivo eran artefactos de exportación y no pacientes; que **LBE** duplica a **LB**, que **DR** es constante y que **Width** equivale exactamente a la diferencia entre **Max** y **Min**, redundancia esta última que hacía singular la matriz de covarianzas e impedía calcular la distancia de Mahalanobis; y que las diez variables comprendidas entre **A** y **SUSP** son una recodificación de la etiqueta **CLASS**. De las 40 columnas iniciales solo 20 son descriptores genuinos.

El estudio de las distribuciones reveló asimetrías extremas y colas pesadas, lo que permitió anticipar, y después confirmar, que los métodos basados en la hipótesis de normalidad rendirían peor que los no paramétricos.

8.2 Sobre el tratamiento de los valores faltantes

La respuesta correcta no consistía en elegir entre la media, la mediana o la moda, sino en reconocer que las filas afectadas no eran observaciones. Las 106 celdas faltantes se concentraban en 3 filas sin diagnóstico, una de ellas una fila de totales cuyos valores quedaban fuera de todo rango fisiológico. Imputarlas con la media habría fabricado tres pacientes ficticios que, además, se habrían convertido en los valores atípicos más extremos del conjunto y habrían contaminado tanto la detección de anomalías como los centroides de K-Means.

El experimento controlado del apartado 3.3 aporta la respuesta a la pregunta general. Cuando los faltantes son reales y dispersos, la mejor estrategia es la imputación por vecinos más cercanos, porque aprovecha la correlación entre variables, y la mediana es la mejor alternativa simple para las variables sesgadas. La media, que suele aplicarse por costumbre, distorsiona la forma de las distribuciones asimétricas, y la moda resulta la peor opción para variables continuas.

8.3 Sobre la detección de anomalías

Isolation Forest es el método elegido. Marcando solo el 5 % de los registros, el 54,2 % de los señalados resultaron ser casos patológicos frente a una tasa base del 8,3 %, lo que supone un enriquecimiento de 6,6 veces y la recuperación de un tercio de todos los casos graves. Su perfil medio, caracterizado por deceleraciones prolongadas y severas, variabilidad errática y bradicardia, reproduce la descripción clínica de un trazado no tranquilizador sin haber visto ni un solo diagnóstico.

Los métodos univariantes quedaron descartados por marcar el 16 % y el 57 % del conjunto respectivamente. El factor local de atipicidad, pese a su sofisticación teórica, fue el peor de los multivariantes, porque en veinte dimensiones la densidad local pierde contraste y porque los casos patológicos de este conjunto no son puntos aislados sino una región periférica poblada que el método interpreta como un vecindario legítimo.

El bajo solapamiento entre métodos, con índices de Jaccard que van de 0,07 a 0,37, demuestra que ser una anomalía es una noción relativa al método empleado. Elegir un detector exige, por tanto, un criterio externo de utilidad; sin él, la elección sería arbitraria.

8.4 Sobre el agrupamiento

K-Means con tres grupos es el modelo elegido, con un índice de Rand ajustado de 0,214 frente a NSP y una prueba chi-cuadrado que descarta el azar. Los tres grupos admiten lectura clínica directa: un grupo tranquilizador de 1 087 registros con un 97,1 % de casos normales, un grupo de vigilancia de 835 registros con un 31 % de sospechosos y un grupo comprometido de 204 registros con un 51,5 % de patológicos, que concentra el 60 % de todos los casos graves del conjunto.

DBSCAN no logró una partición útil porque los datos forman una nube única de densidad decreciente, sin los valles que el algoritmo necesita; sin embargo aisló un grupo de 28 casos íntegramente patológicos y su ruido coincide en un 46 %, medido por el índice de Jaccard, con las anomalías de Isolation Forest. El jerárquico con enlace de Ward produjo una partición equilibrada y coherente, con un índice de 0,179, mientras que los enlaces *average* y *complete* degeneraron pese a exhibir siluetas mucho más altas.

El valor moderado de todos los índices de Rand ajustado tiene una explicación sustantiva y no metodológica: los algoritmos separan bien los extremos, pero la categoría de los sospechosos es por naturaleza una zona de transición y no un fenotipo con frontera propia. Ningún método no supervisado puede recuperar una categoría que no forma un grupo en el espacio de características.

8.5 Conclusión general

El trabajo muestra que el aprendizaje no supervisado recupera estructura clínicamente válida en los datos de cardiocografía. Sin acceder a ningún diagnóstico, los algoritmos identificaron el subconjunto de trazados comprometidos y un gradiente de gravedad coherente con la clasificación obstétrica.

La conclusión práctica es que ambas técnicas resultan complementarias y no alternativas. Isolation Forest responde a la pregunta de qué registros concretos hay que revisar con prioridad, mientras que K-Means responde a la de qué tipos de trazado existen en la población. Un sistema de apoyo a la decisión clínica se beneficiaría de las dos: el agrupamiento para estratificar a la población y la detección de anomalías para priorizar la revisión individual.

La lección metodológica de mayor alcance es que la calidad de un modelo no supervisado no puede juzgarse solo con métricas internas. En dos ocasiones a lo largo de este trabajo el coeficiente de silueta habría conducido a la peor decisión posible, al elegir dos grupos en K-Means y al preferir los enlaces degenerados del agrupamiento jerárquico. El conocimiento del dominio y una referencia externa, cuando existe, resultan insustituibles.

Referencias

- Ayres-de-Campos, D., Bernardes, J., Garrido, A., Marques-de-Sá, J., & Pereira-Leite, L. (2000). SisPorto 2.0: A program for automated analysis of cardiotocograms. *Journal of Maternal-Fetal Medicine*, 9(5), 311-318.
[https://doi.org/10.1002/1520-6661\(200009/10\)9:5<311::AID-MFM12>3.0.CO;2-9](https://doi.org/10.1002/1520-6661(200009/10)9:5<311::AID-MFM12>3.0.CO;2-9)
- Breunig, M. M., Kriegel, H.-P., Ng, R. T., & Sander, J. (2000). LOF: Identifying density-based local outliers. *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 93-104. <https://doi.org/10.1145/342009.335388>
- Calinski, T., & Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics*, 3(1), 1-27. <https://doi.org/10.1080/03610927408827101>
- Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, PAMI-1(2), 224-227.
<https://doi.org/10.1109/TPAMI.1979.4766909>
- Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD-96)*, 226-231.
- Hubert, L., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 2(1), 193-218.
<https://doi.org/10.1007/BF01908075>
- Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. *2008 Eighth IEEE International Conference on Data Mining*, 413-422. <https://doi.org/10.1109/ICDM.2008.17>
- Meneses Yupanqui, A. (2026). *Actividad de Aprendizaje Automático: detección de anomalías y técnicas de agrupamiento sobre el conjunto de datos CTG* [Código fuente]. GitHub.
<https://github.com/amenesesy/actividad-ml-ctg>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53-65.
[https://doi.org/10.1016/0377-0427\(87\)90125-7](https://doi.org/10.1016/0377-0427(87)90125-7)
- Rousseeuw, P. J., & Van Driessen, K. (1999). A fast algorithm for the minimum covariance determinant estimator. *Technometrics*, 41(3), 212-223.
<https://doi.org/10.1080/00401706.1999.10485670>
- Rubin, D. B. (1976). Inference and missing data. *Biometrika*, 63(3), 581-592.
<https://doi.org/10.1093/biomet/63.3.581>
- Tukey, J. W. (1977). *Exploratory data analysis*. Addison-Wesley.
- Ward, J. H. (1963). Hierarchical grouping to optimize an objective function. *Journal of the American Statistical Association*, 58(301), 236-244.
<https://doi.org/10.1080/01621459.1963.10500845>